

AGGREGATES IN DATALOG

A DISSERTATION
SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE
AND THE COMMITTEE ON GRADUATE STUDIES
OF STANFORD UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

Abhijeet Mohapatra
December 2019

© 2019 by Abhijeet Mohapatra. All Rights Reserved.
Re-distributed by Stanford University under license with the author.



This work is licensed under a Creative Commons Attribution-Noncommercial 3.0 United States License.
<http://creativecommons.org/licenses/by-nc/3.0/us/>

This dissertation is online at: <http://purl.stanford.edu/rx626ty8196>

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Michael Genesereth, Primary Adviser

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Chris Re

I certify that I have read this dissertation and that, in my opinion, it is fully adequate in scope and quality as a dissertation for the degree of Doctor of Philosophy.

Richard Waldinger,

Approved for the Stanford University Committee on Graduate Studies.

Patricia J. Gumport, Vice Provost for Graduate Education

This signature page was generated electronically upon submission of this dissertation in electronic format. An original signed hard copy of the signature page is on file in University Archives.

Abstract

Datalog is a logic programming language that is often used as a query language for deductive databases, with many practical applications, ranging from declarative networking to data integration to computational law. While Datalog is highly expressive and easy to understand, there is no agreed-upon way of representing aggregate queries, i.e., queries that provide summary values of sets of data.

Over the years, Datalog has been extended in various ways to address this deficiency. Unfortunately, these extensions are either not expressive enough or do not deal with incremental updates or folding of aggregate queries in a general setting.

We present a Datalog extension, called Datalog_A , that provides a representation for aggregates while eliminating the above weaknesses. We show that Datalog_A is more expressive than prior aggregate extensions of Datalog. Keys to the increased expressivity in Datalog_A are the inclusion of lists and sets as first class citizens in the language, a general aggregation operator that allows us to characterize sets of objects with specified properties, and the use of Prolog-like rules to define aggregation functions on these sets (e.g., count, sum).

We present an algorithm to incrementally maintain materialized Datalog_A views in response to changes to the underlying data. We show that the taken to incrementally maintain Datalog_A views can be decreased by materializing additional views and tracking the number of tuple derivations.

We also present an algorithm to fold Datalog_A queries using views. We characterize two classes of query folding problems for which our algorithm generates maximally contained query answers.

Acknowledgments

I would like to express my deepest gratitude to my advisor, Michael Genesereth, for his invaluable guidance and support throughout my Ph.D. career. He helped me overcome the complacency that stems out of proving that some research problems are intractable and not looking beyond the complexity. Instead, he challenged me to solve hard, seemingly impossible problems by tweaking their setup, analogous to Captain Kirk’s approach in his “Kobayashi Maru” test. For this important lesson, I am indebted to Mike.

I would also like to thank my committee members: Christopher Re, Clark Barrett, Martin Fischer, and Richard Waldinger for helping my defense go smoothly.

I have had the privilege to interact, collaborate with and/or mentor some amazing researchers and students. In particular, I would like to thank Aakanksha Chowdhery, Bertrand Decoster, Dina Safreno, Eric Kao, Ethan Oro, George Younger, Heather Blundell, Jacob Wagner, Kaan Ertas, Max Sutton, Michaela Murray, Robert Chuchro, Ravi Ramamurthy, Robert Ikeda, Samir Menon, Sophia Kivelson, Sudhir Agarwal, and Tristan Krueger.

I would like to thank the administrators and staff at Stanford for their support, especially Carrie Peterson, Claire Stager, Darlene Lazar, Jam Kiattinant, Jill Filice, Meredith Hutchin, and Prachi Balaji.

I am thankful to the Symbium team – Leila Banijamali, Kate Didech, and Phi Phan – for supporting me at my thesis defense. I would also like to express my gratitude to the Banijamali family, especially to Leila. From launching a startup together, to motivating me to finish my dissertation, Leila helped me every step along the way.

Finally, but most importantly, I would like to thank my family. Although, thousands of miles away in the small town of Bhubaneswar, Odisha in India, their best wishes and blessings helped and encouraged me to persevere through my darkest days. Thank you for supporting and encouraging me throughout my Ph.D. journey!

Contents

Abstract	iv
Acknowledgments	v
1 Introduction	1
1.1 Aggregates in Datalog	1
1.2 Contributions	2
1.3 Organization of Thesis	3
2 Background	4
2.1 Datasets	4
2.2 Datalog Programs	4
2.3 Datalog: Semantics	5
2.4 Datalog $\neg$	7
2.5 Built-ins	7
2.6 Query Containment	8
2.7 Query Equivalence	8
2.8 Equivalent Rewritings	8
2.9 Maximally Contained Rewritings	8
3 Datalog_A	10
3.1 Lists in Datalog _A	10
3.2 General Aggregation Operator	12
3.3 Semantics	15
4 Expressive Power of Datalog_A	19
4.1 Aggregates in Datalog $\neg$	19
4.2 Datalog with Stratified Aggregates	20
4.3 Datalog with Monotonic Aggregates	24

4.4	Comparison with Prolog	27
5	Maintenance	29
5.1	Differential Rules for Updating Datalog $\neg$ Views	29
5.2	Differential Rules for Datalog $_A$	30
5.2.1	No Projections in Views	31
5.2.2	Views with Projections	33
5.3	Efficient Incremental Maintenance	34
5.3.1	Time Complexity	34
5.3.2	Optimizations	36
5.4	Conclusions	37
6	Query Folding	39
6.1	Introduction	39
6.2	Inverse Method for Datalog without Aggregates	40
6.3	Inverting Views containing <i>setof</i> subgoals	41
6.3.1	Inverting Views that include Output of <i>setof</i>	42
6.3.2	Inverting Views that do not include Output of <i>setof</i>	44
6.4	Maximally Contained Query Plans	45
6.4.1	Folding a Restricted Class of Queries	46
6.4.2	Foldings that use a Restricted Class of Views	49
6.5	Inverting Views with List Functions and Improvements	51
7	Summary and Future Work	58
7.1	Summary	58
7.2	Future Directions	58
7.2.1	Aggregates in Datalog	58
7.2.2	Efficient Incremental Maintenance	59
7.2.3	Query Folding	59
A	Proofs	61
A.1	Proof of Lemma 3.3.1	61
A.2	Analysis of Worst-Case Time Complexity of Differential Rules	62
B	List Functions	64
	Bibliography	67

List of Tables

5.1	Worst-case time complexity of differential rules for Datalog _A	35
-----	---	----

List of Figures

3.1	Dependency graphs of programs containing <i>setof</i> subgoals	15
3.2	Bottom-up evaluation of programs containing <i>setof</i> subgoals	17
4.1	Comparison of expressive power of languages with aggregates	19
6.1	Graph represented by the <i>edge</i> relation	41
6.2	Folding Datalog _A queries using modified Inverse _{Agg} algorithm	56

Chapter 1

Introduction

Datalog is a logic programming language that is often used as a query language for deductive databases. It has many practical applications, ranging from declarative networking [22, 23, 24] to data integration [10, 12, 21] to computational law [25].

Datalog is highly expressive and easy to understand. Datalog can not only express all positive relational algebra queries [4], but also supports recursive queries. Since Datalog’s syntax is concise, implementations of many practical systems and algorithms in Datalog result in orders of magnitude smaller code than competitive imperative implementations [18].

Unfortunately, there is no agreed-upon way of representing aggregate queries, i.e., queries that provide summary values of sets of data, in Datalog. For example, there is no standard way to define properties such as the count, sum, average of all objects with given properties.

1.1 Aggregates in Datalog

Over the years, Datalog has been extended in various ways to address this deficiency. With the addition of negation-as-failure [38], aggregates such as *min* and *max* can be expressed in Datalog. Extensions of Datalog presented in [1, 7, 31] support aggregate queries using built-in functions, e.g., *sum*, *count*, *average*. Aggregation functions in [31] appear in the body of a Datalog rule, whereas aggregation functions in [1, 7] appear in the head of a Datalog rule. The extension of Datalog presented in [26], called *Datalog-FS*, supports aggregates in recursive Datalog rules.

Although prior extensions of Datalog capture desirable features of aggregates, they all have significant weaknesses. Aggregates such as the shortest path in a graph cannot be expressed in some of these extensions. The domain of aggregation functions in some of the extensions does not include the empty set. As a result, the answers to aggregate queries that are obtained in these extensions are incorrect. Extensions where aggregate functions are included in the head of a rule are not amenable to query rewriting techniques that are commonly used in data integration. In

prior work on aggregates, incremental update of cached aggregate answers is dealt with in a limited setting, e.g., when the queries are non-recursive. Finally, none of these prior extensions support aggregates over power-sets.

1.2 Contributions

In this thesis, we define a Datalog extension, called Datalog_A , that provides a representation for aggregates while eliminating these weaknesses, and we show how to address key questions regarding expressive power, incremental maintenance of aggregate views, and rewriting aggregate queries using views.

There are three key elements in our approach:

1. the inclusion of lists and sets as first class citizens in our language,
2. a general aggregation operator, *setof*, that allows us to characterize sets of objects with specified properties, and
3. the use of Prolog-like rules to define aggregation functions on these sets (e.g. count, sum, average, and so forth).

As an example of our approach to writing aggregate queries, consider a scenario where information about people and their children are stored in a database as a unary relation *person* and a binary relation called *children* respectively. In addition, assume that *length* is a binary function that relates a list to the number of elements in that list. The relationship between people and the number of children they have can be defined as the following Datalog_A query.

$$\text{numchildren}(P, N) \text{ :- } \text{person}(P), \text{ setof}(C, \text{ children}(P, C), S), \text{ length}(S, N)$$

As we shall see, Datalog_A has many desirable properties.

- **Highly expressive aggregate queries.** We show that Datalog_A is more expressive than prior aggregate extensions [1, 7, 26, 31] of Datalog.
- **Incremental maintenance of aggregate views.** We present an algorithm to incrementally update materialized Datalog_A views in response to changes to the underlying data. We also characterize a class of Datalog_A views that can be incrementally updated without accessing the underlying data. In such cases, only the changes to the underlying data and the materialized view are used to compute the updates.
- **Query folding with aggregates.** We present an algorithm, called $\text{Inverse}_{\text{Agg}}$, to rewrite Datalog_A queries using views. We also characterize two classes of folding problems for which

our $\text{Inverse}_{\text{Agg}}$ algorithm generates *maximally contained* query answers. Intuitively, this means that the answers obtained by the $\text{Inverse}_{\text{Agg}}$ algorithm are the best possible answers that one could hope to get using the supplied views.

1.3 Organization of Thesis

We begin this document with background material on Datalog and its semantics in Chapter 2. In Chapter 3, we present Datalog_A and discuss its semantics. In Chapter 4, we show that Datalog_A is strictly more expressive than prior aggregate extensions of Datalog. In Chapter 5, we present an algorithm to incrementally update materialized Datalog_A views. In Chapter 6, we present algorithms to fold Datalog_A queries using views. In Chapter 7, we suggest potential extensions of our work for future research.

Some of the material presented in Chapters 3 – 6 also appears in previous conference publications and technical reports [27, 28, 29, 30].

Chapter 2

Background

2.1 Datasets

The *vocabulary* of a dataset is a collection of *object constants* and *relation constants*. In this thesis, we write constants as strings of lower case letters, digits and underscores, e.g., *length*, 123, and *hello_world*.

Each relation constant has an associated *arity*, i.e., the number of arguments allowed in any expression involving that constant. A *ground term* is an object constant. For example, *a* and *b* are ground terms.

A *ground atom* (or *factoid*) is an expression formed from an n-ary relation constant and n ground terms. For example, if *r* is a binary relation constant and *a* and *b* are object constants, then $r(a,b)$ is a ground atom.

The *Herbrand universe* for a given vocabulary is the set of all ground terms that can be formed from the constants in the vocabulary.

The *Herbrand base* for a dataset is the set of all factoids that can be formed from the constants in its vocabulary. For example, for a vocabulary with just two object constants *a* and *b* and a single binary relation constant *r*, the Herbrand base is $\{r(a,a), r(a,b), r(b,a), r(b,b)\}$.

A *dataset* is any subset of the Herbrand base, i.e. an arbitrary set of the factoids that can be formed from the vocabulary of the dataset.

2.2 Datalog Programs

The vocabulary of a Datalog program is a collection of object constants and relation constants.

Variables allow us to state relationships among objects without naming specific objects. In this thesis, we write variables as strings of alphanumeric characters that begin with a capital letter, e.g., *X*, *Y*, *Z_1*, and so forth.

Atoms are analogous to dataset factoids except that they can optionally contain variables as well as object constants. For example, if r is a binary relation constant, if a is an object constant, and if X and Y are variables, then $r(a, X)$ is an atom, as are $r(a, a)$, $r(X, Y)$, and $r(a, Y)$.

A *Datalog rule* is an expression of the form $H :- B_1, B_2, \dots B_n$ where H is a distinguished atom, called the *head*, and $B_1, B_2, \dots B_n$ are atoms, together called the *body*. Atoms in the body are also called as *subgoals*.

The following expression is an example of a Datalog rule.

$$r(X) :- p(X, Y), q(Y)$$

In the above rule, $r(X)$ is the head, the expression $p(X, Y), q(Y)$ is the body, and $p(X, Y)$ and $q(Y)$ are subgoals.

Informally, a rule is analogous to a reverse implication. For example, the rule above states that r is true of an object X if there is an object Y such that p is true of X and Y and q true of Y .

A *Datalog program* is a set of rules, and a *closed* Datalog program is combination of a Datalog program and a dataset.

2.3 Datalog: Semantics

In the following, we describe the fixpoint semantics of Datalog programs. We start by defining what it means to *apply* a Datalog rule to a dataset.

An *instance* of a rule is a rule in which all variables have been consistently replaced by ground terms.

The application of a rule r to a dataset Δ , denoted as $v(r, \Delta)$, is the set of every atom q such that all of the following conditions hold.

- q is the head of an instance r' of r , and
- every atom in the body of r' is in Δ .

Let Σ denote the rules in a Datalog program. We define the closure $C(\Delta, \Sigma)$ of a dataset Δ with respect to Σ to be the result of repeatedly applying the rules in Σ until fixpoint is reached.

$$\begin{aligned} \Gamma_0 &= \Delta \\ \Gamma_{n+1} &= \bigcup_{r \in \Sigma} v(r, \bigcup_{i \leq n} \Gamma_i) \\ C(\Delta, \Sigma) &= \bigcup_n \Gamma_n \end{aligned}$$

In this case, the *extension* of the closed Datalog program, also referred to as a *model*, is given

by $C(\Delta, \Sigma)$. Every Datalog program has a unique model, which is also the minimal model of the program.

The extension of a relation n -ary relation r in a closed Datalog program is the set of atoms of the form $r(x_1, x_2, \dots, x_n)$ in program's extension.

The fixpoint semantics of Datalog program described above translates naturally into a bottom-up evaluation strategy, commonly referred to as naive evaluation. We illustrate bottom-up evaluation of Datalog programs in the example below.

Example 2.3.1. Consider a dataset Δ that consists of the atoms $edge(a, b)$ and $edge(b, c)$, and a Datalog program Σ that is defined as follows.

$$\begin{aligned} path(X, Y) &:- edge(X, Y) \\ path(X, Y) &:- edge(X, Z), path(Z, Y) \end{aligned}$$

In this case, a fixpoint is reached in four steps, i.e., at $n = 3$.

$$\begin{aligned} \Gamma_0 &= \Delta = \{edge(a, b), edge(b, c)\} \\ \Gamma_1 &= \bigcup_{r \in \Sigma} v(r, \Gamma_0) = \{path(a, b), path(b, c)\} \\ \Gamma_2 &= \bigcup_{r \in \Sigma} v(r, \Gamma_0 \cup \Gamma_1) = \{path(a, b), path(b, c), path(a, c)\} \\ \Gamma_3 &= \bigcup_{r \in \Sigma} v(r, \Gamma_0 \cup \Gamma_1 \cup \Gamma_2) = \{path(a, b), path(b, c), path(a, c)\} \dots \text{fixpoint reached} \end{aligned}$$

Evaluating Σ on the dataset Δ results in the answers, $C(\Delta, \Sigma) = \{edge(a, b), edge(b, c), path(a, b), path(b, c), path(a, c)\}$.

Safety of Datalog Programs. A Datalog rule is *safe* if and only if every variable that appears in the head also appears in an atom in the body. A Datalog program is safe if and only if every rule in the program is safe.

Safety ensures that evaluating a Datalog program results in finite answers. Consider the evaluation of the following unsafe Datalog rule on a dataset containing the factoid $p(a)$.

$$q(X, Y) :- p(X)$$

In this case, since the variable Y is not bound in the body of the rule, there are potentially infinite answers of the form $q(a, y)$.

2.4 Datalog $\neg$

An important practical extension of Datalog is the introduction of negation-as-failure ($\neg$). This enables us to express rules whose bodies contain negation information, e.g., X is a foundational course if X has no prerequisite.

$$\begin{aligned} foundation(C) &:- course(C), \neg requiresprereq(C) \\ requiresprereq(C) &:- prerequisite(C, X) \end{aligned}$$

In this thesis, we refer to the extension of Datalog with negation-as-failure as Datalog $\neg$.

For the purposes of this thesis, it is not important to understand details regarding the semantics of Datalog $\neg$ programs. That said, we note that arbitrary Datalog $\neg$ programs can have *multiple* minimal models. As a result, it is typically required that Datalog $\neg$ programs be *stratified* [38]. We use an analogous restriction in Chapter 3 to ensure unique minimal models of programs with aggregates.

2.5 Built-ins

It is common to extend Datalog to include arithmetic functions (such as plus, times, max, min), string functions (such as concatenation), equality and inequality. A built-in is pre-defined as a relation $r(I_1, I_2, \dots, I_k, O_1, O_2, O_m)$, where $I_1, I_2, \dots, I_k$ and O_1, O_2, O_m refer to the input and the output arguments of the built-in respectively.

The following expression is an example of a Datalog rule that includes the built-in operator *plus*.

$$q(X, Y) :- p(X), plus(X, 1, Y)$$

In the rest of this thesis, we will write built-in atoms using *infix* notation, e.g., $X + 1 = Y$ instead of $plus(X, 1, Y)$, to simplify our presentation.

Safety of Datalog Programs with Built-ins. A Datalog rule containing built-ins is safe if and only if all of the following conditions hold.

- Every variable that appears in the head also appears in a positive atom or as an output argument of a built-in atom in body.
- Every variable that appears as an input argument of a built-in atom in body also appears in a positive atom in the body.

A Datalog program is safe if and only if every rule in the program is safe.

2.6 Query Containment

Definition 2.6.1. A query q_1 is contained in query q_2 if and only if for every dataset D , the set of answers obtained by evaluating q_1 on $D \subseteq$ the set of answers obtained by evaluating q_2 on D .

Consider queries q_1 , q_2 , and q_3 that are defined as follows.

$$\begin{aligned} q_1(X) &:- p(X) \\ q_2(X) &:- p(X), r(X) \\ q_3(X) &:- p(X), r(a) \end{aligned}$$

Clearly, q_2 and q_3 are contained in q_1 . However, q_3 is *not* contained in q_2 .

2.7 Query Equivalence

Definition 2.7.1. A query q_1 is equivalent to query q_2 if and only if for every dataset D , the set of answers obtained by evaluating q_1 and q_2 on D are identical.

Equivalence and containment of query answers are useful when comparing rewritings of queries. If the rewritten query is contained in the original query, then the answers that are obtained by evaluating the rewritten query are *correct*. However, *not* all of the correct answers may be obtained unless the rewritten and the original queries are equivalent.

2.8 Equivalent Rewritings

Definition 2.8.1. A query q_1 is an equivalent rewriting of query q_2 using views V if and only if q_1 is equivalent to q_2 and every relation constant that appears in the body of rules in q_1 is also in V .

In data integration settings, it is often helpful to consider a different notion of containment, called *maximal containment*.

2.9 Maximally Contained Rewritings

Definition 2.9.1. A query q_1 is a maximally contained rewriting of query q_2 in language L using views V if and only if:

- $q_1 \in L$,
- every relation constant that appears in the body of rules in q_1 is also in V ,
- q_1 is contained in q_2 , and

- there does not exist a query $q_3 \in L$ such that q_3 is contained in q_2 and q_1 is contained in q_3 .

Note that if there exists an equivalent rewriting of a query q_2 using views V , then every maximally contained rewriting of q_2 is also equivalent to q_2 .

Chapter 3

Datalog_A

In this chapter, we present an extension of Datalog, called Datalog_A, which supports aggregates under set semantics. Keys to our approach are the introduction of lists as first-class citizens and an aggregation operator, called *setof*, to generate sets. In our presentation, we assume that useful arithmetic, string, and comparison functions are predefined in the underlying Datalog language and their use in the language is restricted by the applicable safety conditions.

3.1 Lists in Datalog_A

Lists are first-class citizens in Datalog_A. The empty list is denoted using the keyword *nil*. To enable us to represent non-empty lists, we introduce a list constructor called *cons*. The constructor *cons* takes two Datalog_A terms *H* and *T* as input, and *cons*(*H*, *T*) denotes the list that is formed by pre-pending *H* to *T*.

Since the second argument to *cons* is a Datalog_A term, the structures that can be represented using *cons* may be non-linear, e.g., trees. To simplify our presentation of Datalog_A, we focus only on linear *cons* structures, which includes unnested lists as well as nested lists. However, all of our results apply to non-linear *cons* structures as well.

A Datalog_A list *cons*(*H*, *T*) may also be specified in an infix format as *H!T*. We may also represent a Datalog_A list by enclosing its elements within square brackets. The term $[]$ denotes *nil*, and $[X_1, X_2, \dots, X_n]$ denotes the list *cons*(*X*₁, *cons*(*X*₂, $\dots$ *cons*(*X*_{*n*}, *nil*)).

The terms $[]$, $[a, b, c]$, $[a, [b]]$, $a![b, c]$, and *cons*(*a*, *cons*(*b*, *nil*)) are examples of syntactically valid Datalog_A lists, and *p*($[]$), *r*(*a*, $[a, b]$), *s*(*X*, *a!Y*) are examples of syntactically valid Datalog_A atoms.

List functions. Inclusion of the list constructor *cons* in Datalog_A enables us to define operations over lists, especially in the presence of arithmetic operators. In the following examples, we show how the list constructor may be used to compute the length of a specified list and to enumerate

the members of a specified list.

Example 3.1.1 (Length of a list). Consider a query q that is defined as follows.

$$\begin{aligned} q(L) &:- \text{length}([a, [b, c], d], L) \\ \text{length}([], 0) \\ \text{length}(H!T, C) &:- \text{length}(H, D), D = C + 1 \end{aligned}$$

The above definition of q leverages a relation called *length* that recursively computes the length of a list. In the base case, the length of the empty list, i.e., $[]$, is 0. The length of a list $H!T$ is recursively defined in terms of length of the sublist T . In query q , length of the list $[a, [b, c], d]$ is 3.

Example 3.1.2 (Membership in a list). We can also enumerate the elements of a list in a manner similar to the definition of *length* in Example 3.1.1 as follows.

$$\begin{aligned} q(X) &:- \text{member}(X, [a, [b, c], d]) \\ \text{member}(X, X!Y) \\ \text{member}(Z, X!Y) &:- \text{member}(Z, Y) \end{aligned}$$

In Datalog_A we refer to relations whose definition involves recursion through lists as *list functions*. In Examples 3.1.1 and 3.1.2, *length* and *member* are examples of list functions. Other list functions that are used in this thesis are defined in Appendix B.

Infinite answers. The gain in expressive power due to the *cons* operator is significant, as the language becomes Turing-complete [2]. In the following example, we show that unrestricted use of *cons* in Datalog_A rules may lead to infinite answers.

Example 3.1.3. Consider a query q that is defined as follows.

$$\begin{aligned} q([X]) &:- q(X) \\ q(a) \end{aligned}$$

In this case, the only model of the program $\{q(a), q([a]), q([[a]], \dots)\}$ is infinite sized. As a result, evaluation of q will not terminate in finite time.

The good news is that we can impose practical restrictions on the use of list functions that are *sufficient* to ensure finite answers and termination. In particular, we use the notion of *admissibility* that was previously proposed in [39], and require that Datalog_A queries be admissible.

Definition 3.1.1 (Admissible Datalog_A queries). A Datalog_A query is admissible if, for every recursive call involving *cons*, there is a non-empty set of bound arguments such that the *size* of at least one of these arguments decreases and the sizes of the other bound arguments do not increase.

The notion of size used in Definition 3.1.1 refers to the size of a term. The size of lists is the number of elements in the list and the size of non-list terms is 0.

Note that the query *q* from Example 3.1.1 is admissible because the first argument of the list function *length* is bound and the size of this argument decreases by 1 in every recursive call. By contrast, the query *q* from Example 3.1.3 is *not* admissible.

Lemma 3.1.1. *Admissible queries have finite answers.*

Proof. This result was proved in [39]. It was also shown in [39] that standard top-down evaluation of admissible programs terminates in finite time. $\square$

3.2 General Aggregation Operator

In the previous section, we introduced lists as first-class citizens and the *cons* operator for constructing lists. In this section, we introduce a general aggregation operator, called *setof*, that generates sets. These sets can subsequently be manipulated using list functions, e.g., *length*.

The *setof* operator has the signature *setof*(*Z*, $\phi(\bar{X})$, *S*). The first argument, *Z*, also called the *aggregation argument*, is a Datalog_A term. The second argument, $\phi(\bar{X})$, also called the *body*, is a conjunction of Datalog_A subgoals which may include additional *setof* subgoals. The third argument, *S*, also called the *output*, is a Datalog_A term.

Intuitively, a Datalog_A atom of the form *setof*(*Y*, $v(x, Y)$, *S*) evaluates to true on a dataset *D* iff *S* is a sorted list of all values *Y* such that $v(x, Y)$ evaluates to true on *D*. For purposes of this thesis, it only matters that the elements in the output of *setof* subgoals be unique and appear in some sorted order, e.g., lexicographic order. However, the choice of sort order is immaterial.

Before we describe the formal semantics of *setof*, we introduce its semantics through the following examples.

Example 3.2.1. Consider a database which consists of a unary base relation *person* and a binary base relation *parent*. Let a tuple of the form *parent*(*x*, *y*) characterize that *x* is the parent of *y*. For every person, we can compute the set of the person's children as follows.

$$children(X, S) :- person(X), setof(Y, parent(X, Y), S)$$

Suppose, a dataset consists of the tuples *person*(*a*), *person*(*b*), and *parent*(*a*, *b*). Evaluating *children* over the dataset results in the answers: *children*(*a*, [*b*]) and *children*(*b*, []).

We can also use the sets generated by the *setof* operator as inputs to list functions to compute aggregates such as *count*, *sum* etc.

Example 3.2.2. Consider the setup in Example 3.2.1. For each person, we compute the number of their children using the list function *length* from Example 3.1.1 in the following query.

$$numchildren(X, C) :- children(X, S), length(S, C)$$

Evaluation of *numchildren* results in the answers: *numchildren(a, 1)* and *numchildren(b, 0)*.

It is often desirable to be able to compute aggregates over multi-sets or *bags*, where elements may be repeated. In the following example, we show how we can effectively aggregate bags using Datalog_A rules under set semantics.

Example 3.2.3 (Aggregating Bags). Consider a database which consists of a unary relation *test* and a ternary relation *score*. Let a tuple of the form *score(x, y, z)* characterize that a student *y* scored *z* marks in test *x*.

Suppose a dataset consists of the tuples *test(t)*, *score(t, s₁, 10)*, *score(t, s₂, 10)*. We can compute the total scores for each test using a query *totalscore* which is defined as follows.

$$\begin{aligned} totalscore(T, A) &:- test(T), setof([M, X], score(T, X, M), S), collectfirst(S, B), sum(B, A) \\ sum([], 0) \\ sum(X!Y, S) &:- sum(Y, B), S = B + X \\ collectfirst([], []) \\ collectfirst(\{[Y, Z]!T, Y!S\}) &:- collectfirst(T, S) \end{aligned}$$

Evaluating *totalscore* on the dataset results in the answer: *totalscore(t, 20)*. Evaluation of *totalscore* first computes the set $[[10, s_1], [10, s_2]]$, which is subsequently converted to the list $[10, 10]$ using the list function *collectfirst*. Finally, the list function *sum* computes the sum (= 20) of the elements in the list $[10, 10]$.

Syntactic Restrictions. We require that all rules containing *setof* subgoals are *safe* and *stratified*. We will discuss the significance of these restrictions after we have formally defined the semantics of Datalog_A programs in Section 3.3.

Definition 3.2.1 (Safety). A rule containing a *setof* subgoal is *safe* *iff* the rule satisfies all of the following conditions.

- Every variable that appears in the head also appears in at least one positive (or non-*setof*) subgoal or in the output argument of a *setof* subgoal.

- Every variable that appears in the body of a *setof* subgoal but not in the aggregation or the output argument of the subgoal also appears in at least one positive subgoal.

The rules shown below are safe.

$$\begin{aligned} q_1(X, S) &:- p(X), \text{ setof}(Y, r(X, Y), S) \\ q_2(X) &:- \text{ setof}(X, p(X), S) \\ q_3(X) &:- \text{ setof}(X, r(a, X), S) \end{aligned}$$

By contrast, the following rule is *not* safe because the variable Y appears in the body of the *setof* subgoal, does not appear in the subgoal's aggregation argument, and does not appear in a positive subgoal.

$$q_4(X) :- p(X), \text{ setof}(Z, r(Y, Z), S)$$

Definition 3.2.2 (Dependency graphs). Given a program containing *setof* subgoals, the dependency graph of the program is a directed graph whose vertices correspond to the relations that appear in the head of a rule in the program, and whose edges satisfy all of the following conditions.

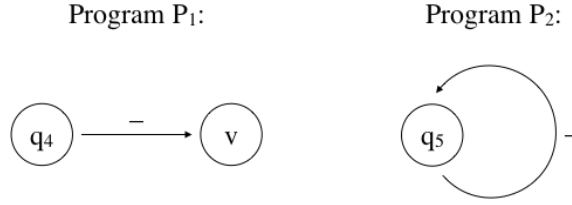
1. For every rule in the program, if h is the relation that appears in the head and p is a positive subgoal, then there exists an edge from h to p .
2. For every rule in the program, if h is the relation that appears in the head and p is a relation that appears in the body of a *setof* subgoal, then there exists an edge from h to p . In addition, this edge is annotated using the ‘ $-$ ’ symbol.

Definition 3.2.3 (Stratified programs). A program containing *setof* subgoals is stratified *iff* the dependency graph of the program does not contain any cycles involving edges annotated with ‘ $-$ ’.

Our definition of dependency graphs for programs containing *setof* subgoals is a straightforward extension of the technique discussed in [38] that is used to test stratification of Datalog $^-$ programs.

Definition 3.2.4 (Stratum). Given a stratified program, stratum of a view relation that appears in the program is the $1 +$ maximum number of ‘ $-$ ’ symbols that appear in a path in the dependency graph of the program starting from the vertex corresponding to the view relation.

The *stratum* mapping partitions the rules of a stratified program into finite strata. Let N be the maximum number of ‘ $-$ ’ symbols that appear in any path in the dependency graph. For $i \in \{1, \dots, N+1\}$, rules in i^{th} stratum are $\bigcup_r \{r \text{ s.t. } p \text{ appears in the head of rule } r \text{ and } \text{stratum}(p) = i\}$.

Figure 3.1: Dependency graphs of programs containing *setof* subgoals

Consider a program P_1 which consists of the following two rules.

$$\begin{aligned} q_4(X, S) &:- p(X), \text{ setof}(Y, v(X, Y), S) \\ v(X, Y) &:- r(X, Z), s(Z, Y) \end{aligned}$$

The dependency graph of P_1 is shown in Figure 3.1. Since there are *no cycles* in the dependency graph of P_1 containing an edge that is annotated with the ‘-’ symbol, the program is stratified. In this case, the rule $v(X, Y) :- r(X, Z), s(Z, Y)$ is in the 1st stratum, whereas the rule $q_4(X, S) :- p(X), \text{ setof}(Y, v(X, Y), S)$ is in the 2nd stratum.

By contrast, consider a program P_2 which consists of the following rule.

$$q_5(X, S) :- p(X), \text{ setof}(X, q_5(X, X), S)$$

The dependency graph of P_2 , shown in Figure 3.1, has a *self-loop* on the vertex q_5 that is annotated with the ‘-’ symbol. As a result, P_2 is *not* stratified.

3.3 Semantics

In this section, we extend the semantics of Datalog programs discussed in Chapter 2 to define the meaning of programs containing *setof* subgoals.

The vocabulary of a Datalog_A program is a collection of object constants, relation constants, *cons*, and $\square$.

The *Herbrand universe* of a Datalog_A program is the set of all ground terms that can be formed using object constants and lists of finite depth and width over those constants.

The *Herbrand base* of a Datalog_A program is the set of all atoms that can be formed from the constants in the vocabulary.

Fixpoint semantics. In the following, we describe the fixpoint semantics of a program containing *setof* subgoals. We start by defining what it means to *apply* a rule to a dataset.

An *instance* of Datalog_A rule is a rule in which all variables that appear in a positive subgoal

have been consistently replaced by ground terms.

The application of a rule r to a dataset Δ , denoted as $v(r, \Delta)$, is the set of every atom q such that all of the following conditions hold.

- q is the head of an instance r' of r ,
- every positive atom of r' is in Δ ,
- for every *setof* atom in r' , $\text{setof}(\bar{Y}, \phi(\bar{X}, \bar{Y}), S)$, $S = \{\bar{Y} \text{ s.t. } \phi(\bar{X}, \bar{Y})\}$.

Given the rules in a stratum Σ , we define the closure $C(\Delta, \Sigma)$ of a dataset Δ with respect to Σ to be the result of repeatedly applying the rules in Σ until fixpoint is reached.

$$\begin{aligned}\Gamma_0 &= \Delta \\ \Gamma_{n+1} &= \bigcup_{r \in \Sigma} v(r, \bigcup_{i \leq n} \Gamma_i) \\ C(\Delta, \Sigma) &= \bigcup_n \Gamma_n\end{aligned}$$

Given a stratified logic program Σ with k -strata, let $\Sigma_1, \dots, \Sigma_k$ denote the rules in the stratum 1 through k respectively. We define the extension $E(\Delta, \Sigma)$ of a dataset Δ with respect to Σ to be the result of applying $\Sigma_1, \dots, \Sigma_k$ to Δ in order.

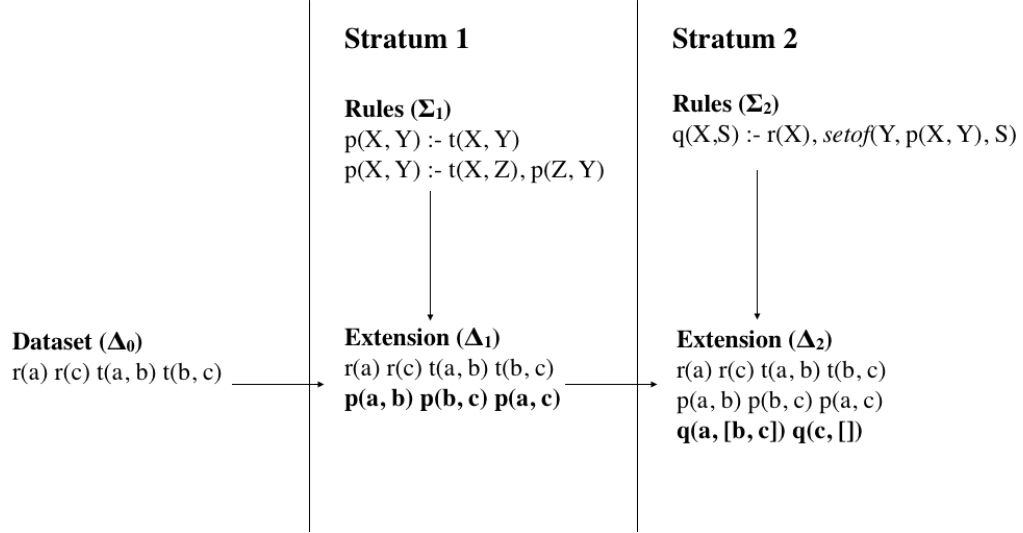
$$\begin{aligned}\Delta_0 &= \Delta \\ \Delta_{i+1} &= \Delta_i \cup C(\Delta_i, \Sigma_{i+1}) \\ E(\Delta, \Sigma) &= \Delta_k\end{aligned}$$

The fixpoint semantics described above translates naturally into a bottom-up evaluation strategy for Datalog_A programs. Note that sets of finite depth and width *need not* be enumerated in the bottom-up evaluation of Datalog_A programs. Instead, the Herbrand universe may initially contain only the ground terms that appear in the dataset or the rules. Starting with the lowest stratum, with each application of rules in the stratum, as atoms are added to the closure, new terms that appear in these atoms are also added to the Herbrand universe. The updated Herbrand universe is used in subsequent applications of rules in the stratum, till a fixpoint is reached.

We illustrate bottom-up evaluation of Datalog_A programs in the following example.

Example 3.3.1. Consider a program Σ which consists of the following rules.

$$\begin{aligned}q(X, S) &:- r(X), \text{ setof}(Y, p(X, Y), S) \\ p(X, Y) &:- t(X, Y) \\ p(X, Y) &:- t(X, Z), p(Z, Y)\end{aligned}$$

Figure 3.2: Bottom-up evaluation of programs containing *setof* subgoals

The program Σ is stratified and its rules can be partitioned into 2 strata. Stratum 1 consists of the rules with p in the head and stratum 2 consists of the rule with q in the head.

Evaluating Σ over the dataset $\Delta = \{r(a), r(c), t(a, b), t(b, c)\}$ results in the extension $\Delta \cup \{p(a, b), p(b, c), p(a, c), q(a, [b, c]), q(c, [])\}$. We show the evaluation steps in Figure 3.2.

Bottom-up evaluation of admissible programs containing *setof* subgoals generates finite answers and terminates in finite time. Using the lemma below, we show that it suffices that we prove the finiteness result for admissible programs where the rules contain *at most* one *setof* subgoal.

Lemma 3.3.1. *A rule containing multiple, potentially nested setof subgoals can be equivalently rewritten using rules that contain at most one setof subgoal.*

Proof. See Appendix A.1. □

Theorem 3.3.1. *Safe, stratified, and admissible Datalog_A programs containing a finite number of subgoals may only generate finite answers and terminate in finite time.*

Proof. Using Lemma 3.3.1, we assume, without loss of generality, that there are *no* rules that contain nested *setof* subgoals in our program. We prove our result for safe and stratified Datalog_A programs with N strata using induction on N .

In the base case, the number of strata in the program is 1, i.e. there are no *setof* subgoals in the program. However, there may be list functions in the program. Since the program is admissible, our result follows directly from Lemma 3.1.1.

Let us assume that admissible Datalog_A programs with $< n$ strata generate finite answers and terminate in finite time. To prove our result using induction, it suffices to prove that admissible

Datalog_A programs with n strata also generate finite answers and terminate in finite time.

Consider an admissible Datalog_A program with n strata. In this program, let Σ_n denote the rules in the n^{th} stratum. Since Lemma 3.1.1 guarantees finite termination of admissible program without *setof* subgoals, it suffices to prove that the *setof* subgoals in Σ_n generate lists of finite size. A *setof* subgoal that appears in Σ_n has the general form *setof*($\bar{X}$, $\phi(\bar{X}, \bar{Y})$, S). For every relation r that appears in ϕ , stratum of r is $< n$. From the induction hypothesis, it follows that evaluation of the extension of ϕ results in finite answers and terminates in finite time. As a result, the size of the list S is finite. $\square$

Regarding use of *setof* and Russel's Paradox. Enabling the definition of sets in a logical language invariably raises the possibility of expressing Russel's Paradox [20]. Fortunately, safety, stratifiability and admissibility of Datalog_A programs ensures that the query which computes the set of all sets that are not members of themselves cannot be expressed in the language. Any attempt to express this query in Datalog_A will result in a call to the list function *member*(X, X) (see Example 3.1.2) or an equivalent thereof, where X is an unbound or free variable, thus violating the admissibility condition from Section 3.1.

Chapter 4

Expressive Power of Datalog_A

In this chapter, we discuss prior work on extending Datalog to support aggregates, and show that the expressive power of Datalog_A is higher than these extensions. Figure 4.1 summarizes the complexity results presented in this chapter. Finally, we compare the expressive power of Datalog_A with Prolog [5].

increasing expressive power ↓	Language \ Property	Lists / Complex terms	Finite Answers	User-defined aggregates
	Datalog	×	✓	×
	Datalog $\neg$	×	✓	×
	Datalog with stratified aggregates [31, 1, 7]	×	✓	×
	Monotonic Datalog: Datalog-FS [26]	×	✓	✓
	Datalog _A	✓	✓	✓
	Prolog	✓	×	✓

Figure 4.1: Comparison of expressive power of languages with aggregates

4.1 Aggregates in Datalog $\neg$

Support for aggregates in Datalog is practically non-existent. At best, one can express queries to determine whether the extension of a relation is *non-empty*. With the addition of the inequality operator ($\neq$), limited forms of counting can be performed in the language, as shown in the following example.

Example 4.1.1. Consider a database that contains a unary relation *person* and a binary relation *friend*. A tuple of the form *friend*(x, y) characterizes that person x is a friend of person y . The

following query finds out persons who have more than 2 friends.

$$q(X) :- \text{person}(X), \text{friend}(X, Y), \text{friend}(X, Z), Y \neq Z$$

Note that the approach used in Example 4.1.1 can be used in general to find answers that have at least k derivations, where k is a constant.

Addition of stratified negation to Datalog extends its ability to support aggregates. The aggregates *min* and *max* can be expressed in Datalog[¬] over ordered domains. Prior work presented in [7] analyzes the incremental change in expressive power when negation, inequalities, and built-in aggregates such as *min*, *count*, *sum*, and *even* are added to Datalog in isolation.

Let $D^\neg$ and D^A be the the set of queries that can be expressed in Datalog[¬] and Datalog_A respectively. In the following, we show that $D^\neg \subset D^A$.

Theorem 4.1.1. $D^\neg \subset D^A$.

Proof. To prove our result, we first show that $D^\neg \subseteq D^A$. Then, we provide an example of a query q , such that $q \in D^A$ and $q \notin D^\neg$.

Without loss of generality, we assume that a query in $D^\neg$ has the following general form.

$$q(\bar{X}) :- \phi(\bar{X}, Y), \neg\psi(\bar{Y})$$

This query can be rewritten in Datalog_A without using the $\neg$ operator as follows.

$$q'(\bar{X}) :- \phi(\bar{X}, Y), \text{setof}(\bar{Y}, \psi(\bar{Y}), [])$$

From the semantics of Datalog_A, it immediately follows that $q'(\bar{x})$ is an answer of q' iff $q(\bar{x})$ is an answer of q .

It was previously shown in [33] that any query that counts the number of tuples in an extension of a relation $\notin D^\neg$. For a supplied relation r , the number of tuples in r 's extension can be counted in Datalog_A as follows.

$$q(C) :- \text{setof}(\bar{X}, r(\bar{X}), S), \text{length}(S, C)$$

Therefore, it follows that $D^\neg \subset D^A$. □

4.2 Datalog with Stratified Aggregates

The extension proposed in [31] added stratified aggregation to Datalog under bag semantics. The proposed language was included in the SQL 3 standard. Aggregates in this extension were supported through a *group-by* operator, which has the signature $\text{group-by}(r(X_1, \dots, X_k), \bar{Y}, [Z_1 =$

$A_1(E_1), \dots, Z_n = A_n(E_n)]$), where:

- r is a k -ary relation,
- $\bar{Y}$ is a sequence *grouping variables* which determines how many aggregate values are computed, and
- A_i is an aggregate function, e.g., *sum* or *count* and E_i is an expression that is formed using $X_1, \dots, X_k$ and other ground terms. A unary function called *distinct* may optionally be applied before applying A_i to deduplicate the values before aggregating.

Informally, the meaning of a rule of the form: $q(X, S) :- \text{group_by}(r(X, Y, Z), [X], [S = f(Y)])$ is as follows. Evaluation of q results in an answer of the form $q(x, s)$ iff there exists bindings of variables Y and Z , such that $r(x, Y, Z)$ is in the extension of r and s is the output of applying the aggregate function f over the bag $\{\{Y \text{ s.t } r(x, Y, Z)\}\}$. In the following example, we illustrate the use of *group_by* operator to support aggregate queries. Details regarding the semantics of *group_by* can be found in [31].

Example 4.2.1. Consider a database that consists of a ternary relation r . The following query computes the sum of the bag of values of the second argument (Y) for every binding of the first argument (X) in r .

$$q(X, A) :- \text{group_by}(r(X, Y, Z), [X], A = \text{sum}(Y))$$

Evaluating q over a dataset containing the tuples $r(a_1, 10, b_1)$, $r(a_1, 10, b_2)$, and $r(a_2, 10, b_2)$ results in the answers $q(a_1, 20)$ and $q(a_2, 10)$.

Note that if using $\text{sum}(\text{distinct}(Y))$ instead of $\text{sum}(Y)$ in the query would result in the answers $q(a_1, 10)$ and $q(a_2, 10)$.

The Datalog extension presented in [31] imposed a stratification condition [31] on Datalog programs containing *group_by* subgoals which prevents recursion through *group_by* operator. This restriction was applied to ensure that programs have unique minimal models and terminate in finite time.

Variations of language presented in [31] were proposed in [1, 7]. Aggregate functions in these extensions included in the *head* of the rule in contrast to [31] where the *group_by* subgoal is included in the *body* of a rule. For example, the query from Example 4.2.1 can be expressed using the language presented in [1] as follows.

$$q(X, \text{sum}(Y)) :- r(X, Y, Z)$$

In fact, it has been shown in [7] that the set of queries that can be expressed using the stratified

extensions [1, 7, 31] are identical. Let us denote the set of queries that are expressible using these extensions as D^{str} . In the following, we show that any query $q \in D^{str}$ can be expressed in Datalog_A.

Lemma 4.2.1. $D^{str} \subseteq D^A$.

Proof. To prove that $D^{str} \subseteq D^A$, it suffices to show that the *group_by* operator [31] can be expressed in Datalog_A. Without loss of generality let us assume that a query in $D^\top$ has the following general form.

$$q(\bar{X}, A) :- \text{group_by}(\phi(\bar{X}, \bar{Y}), \bar{X}, A = f(E))$$

In q 's definition, f is an aggregation function, e.g., *sum*, *count*, *min*, and E is an expression over the terms appearing in $\phi(\bar{X}, \bar{Y})$. Evaluation of q results in an answer of the form $q(\bar{x}, a)$ iff there exists bindings of variables in $\bar{Y}$, such that $\phi(\bar{x}, \bar{Y})$ is an answer of ϕ and a is the output of applying the aggregate function f over the bag $\{\{E \text{ s.t. } \phi(\bar{x}, \bar{Y})\}\}$.

Let V be a list of variables $[Y_1, \dots, Y_k]$ that appear in $\bar{Y}$ but *do not* appear in E . For $f = \text{sum}$, we rewrite q in Datalog_A using the list functions *collectfirst* and *sum* that were defined in Example 3.2.3 as follows.

$$\begin{aligned} q'(\bar{X}, A) &:- v_1(\bar{X}), \text{setof}(E!V, \phi(\bar{X}, \bar{Y}), T), \text{collectfirst}(T, S), \text{sum}(S, A) \\ v_1(\bar{X}) &:- \phi(\bar{X}, \bar{Y}) \end{aligned}$$

We can generalize our rewriting approach to other aggregation functions, e.g., *count*, *min* by defining them as list functions, arithmetic and comparison operators in Datalog_A (see Appendix B).

Suppose that evaluation of q' results in an answer $q'(\bar{x}, a)$. This implies that $v_1(\bar{x})$ is answer of the query v_1 and a is the sum of the terms in the bag $\{\{E \text{ s.t. } \phi(\bar{x}, \bar{Y})\}\}$. Therefore, $q'(\bar{x}, a)$ is an answer to q' iff $q(\bar{x}, a)$ is an answer to q . $\square$

In the following, we give an example of our approach to rewriting aggregate queries in D^{str} .

Example 4.2.2. Consider a database containing binary relations p and r , and a query over this database that is defined using the Datalog extension presented in [1] as follows.

$$q(X, \text{count}\langle Z \rangle) :- p(X, Y), r(Y, Z)$$

This query can be rewritten in Datalog_A as follows.

$$\begin{aligned} q'(X, C) &:- v(X), \text{setof}([Z, Y], p(X, Y), r(Y, Z), S), \text{collectfirst}(T, S), \text{length}(S, C) \\ v(X) &:- p(X, Y), r(Y, Z) \end{aligned}$$

A limitation of stratified aggregation, identified in [33], is that it *cannot* express the *summarized explosion query* which is defined as follows.

Definition 4.2.1 (Summarized explosion query). Consider a database containing a ternary relation *subpart* over the integer domain. A tuple *subpart*(*x*, *y*, *z*) characterizes that a part *x* in a car consists of *C* direct sub-parts *y*. How many copies of sub-part *y* are needed to construct one copy of part *p*?

In the following, we show that the summarized explosion query $\in D^A$.

Lemma 4.2.2. *Summarized explosion query* $\in D^A$.

Proof. First, we define a relation called *requires*(*X*, *Y*, *N*, *P*) which computes the number of copies *N* of component *Y* that *X* requires through a *derivation chain* *P*. A derivation chain is a list of parts $x_1, x_2, \dots, x_k$ such that $\forall i, 1 \leq i < k$, *subpart*(x_i, x_{i+1}, c_i) appears in the extension of *subpart*. The number of copies $N = \prod_i c_i$.

$$\begin{aligned} \text{requires}(X, Y, N, P) &:- \text{paths}(P), \text{first}(X, P), \text{last}(Y, P), \text{summarize}(P, N) \\ \text{paths}(P) &:- \text{setof}(X, \text{part}(X), S), \text{sublist}(S, T), \text{permute}(T, P) \\ \text{summarize}([], 0) \\ \text{summarize}([X], 1) \\ \text{summarize}(X!Y!Z, C) &:- \text{subpart}(X, Y, A), \text{summarize}(Y!Z, B), C = A \times B \end{aligned}$$

The list functions *sublist* and *permute* generate the sub-lists and permutations of supplied list respectively. The list functions *first* and *last* compute the first and the last elements of a supplied list respectively. These list functions are defined in Appendix B.

We can express the summarized explosion query using the *requires* relation as follows.

$$\begin{aligned} \text{numcopies}(X, Y, C) &:- \text{copies}(X, Y, T), \text{collectfirst}(T, S), \text{sum}(S, C) \\ \text{copies}(X, Y, T) &:- \text{part}(X), \text{part}(Y), \text{setof}(N!P, \text{requires}(X, Y, N, P), T) \end{aligned}$$

Evaluation of *numcopies* results in answers of the form *numcopies*(*x*, *y*, *c*) iff $c = \sum_{p(x,y)} N_{p(x,y)}$, where *p*(*x*, *y*) is a derivation chain $[x, e_1, e_2, \dots, e_k, y]$ and $N_p(x, y) = c_x \times c_y \times \prod_i c_i$ such that *subpart*(x, e_1, c_x), *subpart*(e_1, e_2, c_1), ..., *subpart*(e_{k-1}, e_k, c_{k-1}), and *subpart*(e_k, y, c_y) appear in the extension of *subpart*. $\square$

Theorem 4.2.1. $D^A \subset D^{str}$.

Proof. Follows directly from Lemmas 4.2.1 and 4.2.2. $\square$

4.3 Datalog with Monotonic Aggregates

To overcome the limitations of stratified aggregation, prior work in [26] proposed an extension of Datalog, called *Datalog-FS*. Datalog-FS can express queries with aggregates that are monotonic with respect to set-containment, e.g., summarized explosion query.

The key to the increased expressivity of Datalog-FS is due to the introduction of *running-FS* goals which can track the number of distinct derivations of an atom or a conjunction of atoms. A running-FS goal has the signature $K : [\phi(\bar{X})]$, where $\phi(X)$ is a conjunction of atoms and K is a positive number or a variable that does not appear in $\phi(\bar{X})$.

Let us illustrate the use of *running-FS* goals to express aggregate queries.

Example 4.3.1. Consider the query presented in Example 4.1.1. This query can be expressed in Datalog-FS as follows.

$$q(X) \text{ :- } person(X), 2 : [friend(X, Y)]$$

Informally, we can think of the running-FS goal $2 : [friend(X, Y)]$ as a compact way to express: $friend(X, Y), friend(X, Z), Y \neq Z$. This running-FS goal is also equivalent to the expression: $friend(X, Y_1), 1 : [friend(X, Y_2), Y_2 \neq Y_1]$.

If there is *no* recursion through running-FS goals, there is a unique minimal model and the evaluation method presented in [26] computes it. Even in the case where a Datalog-FS program involves recursion through running-FS goals, there is a unique minimal model [26] of the program.

The following example, derived from [26], shows how Datalog-FS can express an unstratified aggregate which has a unique minimal model.

Example 4.3.2 (*K-Party Invitations*). Consider a database consisting of unary relation *sure* and binary relations *friend* and *requires*. A tuple of the form *sure*(*x*) characterizes that person *x* is sure to come to the party. A tuple of the form *friend*(*x*, *y*) characterizes that *x* is a friend of *y*. A person *y* will attend the party if at least *k* of *y*'s friends will attend as well. This information is characterized using tuples of the form *requires*(*y*, *k*). The query – *who will attend the party?* – can be expressed in Datalog-FS as follows.

$$\begin{aligned} attend(X) &\text{ :- } sure(X) \\ attend(X) &\text{ :- } requires(X, K), K : [friend(X, Y), attend(X)] \end{aligned}$$

Note that the above query has a unique *minimal* model. Suppose our dataset (Δ) consists of the following tuples: *friend*(*a*, *b*), *friend*(*b*, *a*), *requires*(*a*, 1), and *requires*(*b*, 1). In this case, Δ is the *only* minimal model. However, this model may not be the *only* intuitive model. Persons *a* and *b* may decide to attend the party together, in which case the model $\Delta \cup \{attend(a), attend(b)\}$ would

be the expected extension of the program. Interestingly enough, in the next section, we will show that *all* models of this query can be expressed in Datalog_A.

Let D^{FS} denote the set of queries that can be expressed in Datalog-FS. In the following, we show that all queries in D^{FS} can be expressed in Datalog_A.

Lemma 4.3.1. $D^{FS} \subseteq D^A$.

Proof. To prove $D^{FS} \subseteq D^A$, it suffices to show that a running-FS aggregate can be expressed using Datalog_A. Without loss of generality, let us assume that a Datalog-FS query containing a running-FS aggregate has following general form.

$$q(\bar{X}) :- \phi(\bar{X}), K : [\psi(\bar{X}, \bar{Y})]$$

In the following we will show how this query can be rewritten in Datalog_A. First, we consider the case when there is *no recursion* through the running-FS aggregate, i.e., ψ is *not* defined using q . In this case, q can be expressed in Datalog_A using the list function *length* that was defined in Example 3.1.1 as follows.

$$q'(\bar{X}) :- \phi(\bar{X}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S), \text{length}(S, C), C \geq K$$

Now, let us consider the case when there is *recursion* through the running-FS aggregate, i.e., the definition of ψ involves q . Note that, for this case, we *cannot* use the rewriting strategy that we used in q' above. This is because the resulting Datalog_A program will be *unstratified*. In such cases, we can use Algorithm 4.1, described below, to rewrite a Datalog-FS query in Datalog_A. The list functions *sublist* and *length* that are used in Algorithm 4.1 are defined in Appendix B.

Note that if $q(\bar{x})$ is an answer of the query q , then $\phi(\bar{x})$ is also an answer of the query ϕ . Algorithm 4.1 computes *all* subsets S of $\{\bar{X} \text{ s.t. } \phi(\bar{X})\}$ in Step 1 and filters out sets that do not satisfy the running-FS aggregate (Steps 1–2). To avoid recursion through *setof* subgoals, for each set S , the answers of ψ are computed by effectively treating q as a base relation whose extension contains the set of tuples in S . This is done in Steps 3–4 of the algorithm where occurrences of $q(\bar{X})$ in the body of a rule are replaced with calls to *member*(X, S).

Let A be the set of the sets that are *not* filtered out, and A_m be a *minimal* set of A , i.e., $A_m \in A$ and for every subset S of A_m , $S \not\subseteq A$. Since, Datalog-FS queries have a unique minimal model with respect to set containment [26], A_m is *guaranteed* to be unique. A tuple $q'(\bar{x})$ is an answer of q' *iff* $q'(\bar{x}) \in A_m$. This implies that, $q'(\bar{x})$ is an answer of q' *iff* $q(\bar{x})$ is an answer of q . $\square$

Consider the application of Algorithm 4.1 to rewrite the Datalog-FS query from Example 4.3.2.

Algorithm 4.1 Rewriting Datalog-FS queries in Datalog_A

Input: Datalog-FS query $q(\bar{X}) :- \phi(\bar{X}), K : [\psi(\bar{X}, \bar{Y})]$, where relations in ψ may be defined using q .

Let R be the set of Datalog_A rules, initially empty.

Step 1: Define relations $answers_q$ and bad and add these definitions to R .

$$\begin{aligned} all(S) &:- setof(\bar{X}, \phi(\bar{X}), T), sublist(T, S), \neg bad(S) \\ bad(S) &:- member(X, S), \neg good(X, S) \end{aligned}$$

Step 2: Define the relation $good$ as $good(\bar{X}, S) :- \phi(\bar{X}), setof(\bar{Y}, \psi(\bar{X}, \bar{Y}), T), length(T, C), C \geq K$, and add this definition to R .

Step 3: Add the definitions of the relations in ψ to R .

Step 4: Let S be a variable that does not appear in any rule $\in R$. Apply the following changes to every rule $\in R$.

1. Let C be a strongly connected component in the dependency graph of the input Datalog-FS program that contains q . For every relation r in $C \setminus \{q\}$, replace atoms of the form $r(\bar{X})$ in R with $r(\bar{X}, S)$.
2. Replace every occurrence of $q(\bar{X})$ in the body of a rule with $member(\bar{X}, S)$.

Output: $R \cup \{q'(\bar{X}) :- all(S), setof(Y, sublist(S, Y), all(Y), []), member(\bar{X}, S)\}$

The resulting Datalog_A query is presented below.

$$\begin{aligned} attend'(X) &:- all(S), setof(Y, sublist(S, Y), all(Y), []), member(\bar{X}, S) \\ all(S) &:- setof(\bar{X}, v(X), T), sublist(T, S), \neg bad(S) \\ bad(S) &:- member(X, S), \neg good(X, S) \\ v(X) &:- sure(X) \\ v(X) &:- requires(X, K) \\ good(X, S) &:- sure(X) \\ good(X, S) &:- requires(X, K), setof(Y, friend(X, Y), member(Y, S), T), length(T, C), C \geq K \end{aligned}$$

Consider the evaluation of the above program on a dataset (Δ) containing the tuples: $friend(a, b)$, $friend(b, a)$, $requires(a, 1)$, and $requires(b, 1)$. The minimal model of this closed program is $\Delta \cup \{v(a), v(b), all([], all([a, b]), good(a, [a, b]), good(b, [a, b]), bad([a]), bad([b])\}$.

The extension of $attend'$ is empty. This is expected since the extension of q from Example 4.3.2 was also empty. That said, the extension of all characterizes *all possible* answers of the query. If we replace the *setof* subgoal in $attend'$ with $setof(Y, all(Y), \neg sublist(S, Y), [])$, evaluating $attend'$ would result in the answers: $attend'(a)$ and $attend'(b)$, which corresponds to the scenario when persons a and b decide to attend the party together.

Lemma 4.3.2. $D^A \not\subseteq D^{FS}$.

Proof. Consider a course planning query which is defined as follows.

Definition 4.3.1 (Course planning). Given n courses $c_1, c_2, \dots, c_n$, each of which may only be taken for u_i units for $1 \leq i \leq n$, is there a selection of courses whose units total k units?

The above query *cannot* be expressed in Datalog-FS. This is because the query is *not* monotonic with respect to set containment.

In the following, we express the course planning query in Datalog_A. We assume that the course information is encoded using a binary relation *units* whose extension consists of the tuples of the form *units*(c_i, u_i) for $1 \leq i \leq n$.

$$\begin{aligned} \text{selectionexists} &:- \text{all}(S), \text{setof}(U!X, \text{member}(X, S), T), \text{collectfirst}(T, L), \text{sum}(L, k) \\ \text{all}(S) &:- \text{setof}(C, \text{course}(C), T), \text{sublist}(T, S) \\ \text{course}(C) &:- \text{units}(C, U) \end{aligned}$$

The list functions *member*, *collectfirst*, *sum*, and *sublist* used in the above query are defined in Appendix B. □

Theorem 4.3.1. $D^{FS} \subset D^A$.

Proof. Follows directly from Lemmas 4.3.1 and 4.3.2. □

4.4 Comparison with Prolog

The logic programming language Prolog [5] also provides support for aggregates. Not only does it support lists and other functional terms in the language but also allows arbitrary recursion through these terms and has two generalized aggregation operators called *bagof* and *setof_p*. As a result, there are queries that can be expressed in Prolog that are not expressible in Datalog_A. But, on the flip side, Prolog queries, unlike Datalog_A queries, are *not* guaranteed to terminate. In the following, we show that Prolog's aggregation operators *bagof* and *setof_p* can be expressed in Datalog_A.

Prolog's *bagof* operator has the signature *bagof*($\bar{Y}, \bar{Z}^{\wedge} \phi(\bar{X}, \bar{Y}, \bar{Z}), S$), where ϕ is a formula over the sequences of terms $\bar{X}$, $\bar{Y}$, $\bar{Z}$, and $\bar{W}$. Consider a Prolog query that is defined as follows.

$$q(\bar{X}, S) :- \text{bagof}(\bar{Y}, \bar{Z}^{\wedge} \phi(\bar{X}, \bar{Y}, \bar{Z}), S)$$

Evaluation of the above query results in an answer $q(\bar{x}, s)$ *iff* there exist bindings of $\bar{Y}_1$ and $\bar{Z}_1$ such that $\phi(\bar{x}, \bar{Y}_1, \bar{Z}_1)$ and s is the bag of terms $\{\{\bar{Y} \text{ s.t. } \exists \bar{Z} \phi(\bar{x}, \bar{Y}, \bar{Z})\}\}$.

Prolog's *setof_p* operator has a signature that is similar to *bagof*. Suppose *setof_p* was used in place of *bagof* in the definition of q above. In this case, evaluation of q results in an answer $q(\bar{x}, s)$ *iff* there

exist bindings of $\bar{Y}_1$ and $\bar{Z}_1$ such that $\phi(\bar{x}, \bar{Y}_1, \bar{Z}_1)$ and s is the set of terms $\{\bar{Y} \text{ s.t. } \exists \bar{Z} \phi(\bar{x}, \bar{Y}, \bar{Z})\}$.

Expressing *bagof* in Datalog_A. Consider a Prolog query that is defined over a base relation p as follows.

$$q(X, S) :- \text{bagof}(Z, Y^{\wedge} p(X, Y, Z), S)$$

This query can be expressed in Datalog_A using the list function *collectfirst* that was defined in Example 3.2.3 as follows.

$$\begin{aligned} q'(X, S) &:- v_1(X), \text{setof}([Z, Y], p(X, Y, Z), T), \text{collectfirst}(T, S) \\ v_1(X) &:- p(X, Y, Z) \end{aligned}$$

Suppose evaluation of q' above results in an answer $q'(x, s)$. This implies that $v_1(x)$ is an answer to the query v_1 , and $s =$ the bag of terms $\{Z \text{ s.t. } \exists Y p(x, Y, Z)\}$. In other words, $q'(x, s)$ is an answer *iff* $\exists Y, Z p(x, Y, Z)$ and $s =$ the bag of terms $\{Z \text{ s.t. } \exists Y p(x, Y, Z)\}$. Therefore, $q'(x, s)$ is an answer of q *iff* $q(x, s)$ is an answer of q .

Expressing *setof_p* in Datalog_A. Suppose we modify the Prolog query q that was defined above by replacing *bagof* with *setof_p*. The modified query $q_1(X, S) :- \text{setof}_p(Z, Y^{\wedge} p(X, Y, Z), S)$ can be expressed in Datalog_A as follows.

$$\begin{aligned} q'_1(X, S) &:- v_2(X), \text{setof}(Y, v_3(X, Y), S) \\ v_2(X) &:- p(X, Y, Z) \\ v_3(X, Y) &:- p(X, Y, Z) \end{aligned}$$

Suppose evaluation of q'_1 above results in an answer $q'_1(x, s)$. This implies that $v_2(x)$ is an answer to the query v_2 and s is the set of terms $\{Y \text{ s.t. } v_3(x, Y)\}$. In other words, $q'_1(x, s)$ is an answer *iff* $\exists Y, Z p(x, Y, Z)$ and s is the set of terms $\{Y \text{ s.t. } \exists Z p(x, Y, Z)\}$. Therefore, $q'_1(x, s)$ is an answer of q *iff* $q_1(x, s)$ is an answer of q_1 .

Chapter 5

Maintenance

Views are often materialized in a database to support efficient querying of the data. A materialized view becomes out-of-date when the underlying base relations from which the view is derived are changed. In such cases, the materialized view is either recomputed from the base relations or the changes in the base relations are incrementally propagated to the view to ensure the correctness of the answers to queries against the view. Prior work presented in [36, 32] shows that incrementally maintaining a materialized view can be significantly faster than recomputing the view from the base relations especially if the size of the view is large compared to the size of the changes.

Several techniques [6, 13, 15, 16, 32, 34, 36] have been proposed to incrementally maintain views in response to changes to the base relations. However, only a fraction of the prior work on incremental view maintenance [6, 16, 32, 36] addresses the maintenance of views that contain aggregates. Furthermore, most of this work addresses maintenance of aggregates in limited settings, e.g., cases where aggregate view's definition includes at most one aggregation operator, or where the aggregation operator used may only be one of *min*, *max*, *sum*, or *count*.

In this chapter, we extend the *differential rules* approach presented in [34] to incrementally maintain Datalog_A views. Instead of focusing on maintenance of *built-in* aggregates like *max* or *sum*, our approach focuses on incrementally maintaining views that are defined using the general aggregation operator – *setof*.

5.1 Differential Rules for Updating Datalog[¬] Views

The approach presented in [34] to incrementally maintain Datalog[¬] views is similar in principle to differentiation in calculus. The changes to the view relations, referred to as *differential relations*, are defined using view relations, base relations, and the changes to view and base relations.

Definition 5.1.1 (Differentials). For any relation p , the differential relations p^+ and p^- characterize

the set of p tuples that are inserted into or deleted from p 's extension respectively. Given an instance of p , and the differentials p^+ and p^- , the updated extension of p is computed as $(p \setminus p^-) \cup p^+$.

The technique presented in [34] incrementally maintains materialized Datalog $\neg$ views using differentials. We show an application of these differential rules in the example below.

Example 5.1.1 (Differential Rule). Consider a materialized view v which is defined by joining the relations p and r as follows.

$$v(X, Z) :- p(X, Y), r(Y, Z)$$

Using the approach presented in [34], the following differential rules can be used to incrementally maintain v in response to additions of the extensions of p and r .

$$v^+(X, Z) :- p(X, Y), r^+(Y, Z)$$

$$v^+(X, Z) :- p^+(X, Y), r(Y, Z)$$

$$v^+(X, Z) :- p^+(X, Y), r^+(Y, Z)$$

Consider an addition of $p(a_1, b_2)$ to the extension of p ; this addition would be characterized using the differential $p^+(a_1, b_2)$. Suppose that, before this addition, the extensions of p , r , and v contained the tuples: $p(a_1, b_1)$, $r(b_1, c_1)$, $r(b_2, c_2)$, and $v(a_1, c_1)$. In response to the addition of $p(a_1, b_2)$, the differential of v would be computed as $v^+(a_1, c_2)$ and the updated extension of v would contain $v(a_1, c_1)$ and $v(a_1, c_2)$.

5.2 Differential Rules for Datalog $_A$

In this section, we will extend the differential rules presented in [34] to compute differentials of Datalog $_A$ views. Consider a materialized view v that is defined over a unary relation p , a binary relation r using the list function *length* as follows.

$$v(X, A) :- p(X), \text{setof}(Y, r(X, Y), S), \text{length}(S, A)$$

We can equivalently rewrite v using an auxiliary view u as follows.

$$v(X, A) :- u(X, S), \text{length}(S, A)$$

$$u(X, S) :- p(X), \text{setof}(Y, r(X, Y), S)$$

Incremental maintenance of the auxiliary view, which contains *setof* subgoals but does not contain any list functions, is key to the maintenance of v . Once we compute the updates to u , we can apply the list functions to the updated tuples to compute the new extension of v .

Before we proceed, we make a simplifying assumption that the view is defined using exactly *one* *setof* subgoal. This assumption *does not* limit the applicability of our approach to views with multiple *setof* subgoals (see Lemma 3.3.1). We also assume that the variables in the output argument of the *setof* subgoal are included in the head of the view. We lift this restriction later in Section 5.3.2.

Differentiating views containing one *setof* subgoal. There are two possible ways in which a view v can be defined in Datalog_A using one *setof* subgoal.

- **Case 1 (No Projections).** In this case, all of the variables that are bound outside the *setof* subgoal are included in the head of the view.
- **Case 2 (With Projections).** In this case, not all of the variables that are bound outside of the *setof* subgoal are included in the head of the view.

5.2.1 No Projections in Views

Consider a view v that is defined over the formulas $\phi(\bar{X})$ and $\psi(\bar{X}, \bar{Y})$.

$$v(\bar{X}, S) :- \phi(\bar{X}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)$$

To facilitate our presentation of differential rules for v^+ and v^- , we define a view ψ_u as follows.

$$\psi_u(\bar{X}) :- \psi^+(\bar{X}, \bar{Y})$$

$$\psi_u(\bar{X}) :- \psi^-(\bar{X}, \bar{Y})$$

The following differential rules compute the differentials v^+ and v^- as views over v , ϕ , ψ , and the differentials ϕ^+ , ϕ^- , ψ^+ , and ψ^- .

$$v^-(\bar{X}, S) :- \phi^-(\bar{X}), v(\bar{X}, S) \tag{\Delta_1}$$

$$v^+(\bar{X}, S) :- \phi^+(\bar{X}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S_1), \text{setof}(\bar{Y}, \psi^+(\bar{X}, \bar{Y}), S_2), \\ \text{setof}(\bar{Y}, \psi^-(\bar{X}, \bar{Y}), S_3), S = (S_1 \setminus S_3) \cup S_2 \tag{\Delta_2}$$

$$v^+(\bar{X}, S) :- \psi_u(\bar{X}), v(\bar{X}, S_1), \neg\phi^+(\bar{X}), \neg\phi^-(\bar{X}), \text{setof}(\bar{Y}, \psi^+(\bar{X}, \bar{Y}), S_2), \\ \text{setof}(\bar{Y}, \psi^-(\bar{X}, \bar{Y}), S_3), S = (S_1 \setminus S_3) \cup S_2 \tag{\Delta_3}$$

$$v^-(\bar{X}, S) :- \psi^+(\bar{X}, \bar{Y}), v(\bar{X}, S) \tag{\Delta_4}$$

$$v^-(\bar{X}, S) :- \psi^-(\bar{X}, \bar{Y}), v(\bar{X}, S) \tag{\Delta_5}$$

Correctness. In the following, we prove the correctness of the differential rules $\Delta_1 - \Delta_5$.

Lemma 5.2.1. *The differential rules $\Delta_1 - \Delta_5$ correctly maintain a view containing a *setof* subgoal where all the variables that are bound outside the *setof* subgoal are included in the head of the view.*

Proof. Suppose the tuple $\phi(\bar{x})$ is deleted from the extension of ϕ . In this scenario, the tuple $v(\bar{x}, s)$ in the extension of v should be deleted. This update is captured by the differential rule Δ_1 .

Suppose the tuple $\phi(\bar{x})$ is inserted into the extension of ϕ . In this scenario, the tuple $v(\bar{x}, s)$ should be *inserted* into the extension of v , where $s = \{\bar{y} \mid \neg\psi(\bar{x}, \bar{y}) \wedge (\psi(\bar{x}, y) \vee \psi^+(\bar{x}, y))\}$. This update is captured by the differential rule Δ_2 .

Suppose the extension of ϕ *does not* change. In this scenario, *no* new tuples should be inserted into or deleted from the extension of v , but, existing tuples may be *updated*. More specifically, the set argument in existing tuples may be updated based on updates to the extension of ψ . Suppose the extension of v contains the tuple $v(\bar{x}, s)$, $s_i = \{\bar{y} \mid \psi^+(\bar{x}, \bar{y})\}$, and $s_d = \{\bar{y} \mid \psi^-(\bar{x}, \bar{y})\}$. Then, as a result of the update, the tuple $v(\bar{x}, s)$ should be updated to $v(\bar{x}, s')$, where $s' = (s \setminus s_d) \cup s_i$. Updates of this form are captured by the differential rules $\Delta_3 - \Delta_5$. $\square$

Example 5.2.1. Consider a view v that is defined over the unary relation p and the binary relation p as follows.

$$v(X, S) :- p(X), \text{ setof}(Y, r(X, Y), S)$$

The extensions of p and r consists of the tuples: $p(a), p(b), r(a, 1)$ and $r(c, 3)$ and the materialization of v contains the tuples $v(a, [1])$ and $v(b, [])$.

Suppose the tuple $p(a)$ is deleted from the extension of p and the tuple $r(b, 2)$ is added to the extension of b . These changes can be characterized using the differentials $p^-(a)$ and $r^+(b, 2)$.

By applying $\Delta_1 - \Delta_5$, we compute the differentials of v as $v^+(b, [2])$, $v^-(b, [])$, and $v^-(a, [1])$. As a result, the tuple $v(a, [1])$ is deleted from the extension of v and the tuple $v(b, [])$ is updated to $v(b, [2])$.

Self-maintainability. Views without projections can be incrementally maintained without accessing the underlying base relations. This property called self-maintainability [19] is highly desirable when access to the base relations is restricted, when the base relations themselves are hypothetical, e.g., in a LAV integration scenario [38], or when the size of the base relations is very large compared to the size of the view.

Corollary 5.2.1 (Self-maintainability). *Materialized views that have the general form $v(\bar{X}, S) :- \phi(\bar{X}), \text{ setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)$, where ϕ and ψ are base relations, can be incrementally maintained in response to changes to the extension of ψ without accessing the extensions of ϕ and ψ .*

Proof. This result directly follows from the fact that the relations ϕ and ψ *do not* appear in the body of the differential rules $\Delta_3 - \Delta_5$. $\square$

5.2.2 Views with Projections

Consider a view v that is defined over the formulas $\phi(\bar{X}, \bar{Y})$ and $\psi(\bar{X}, \bar{Y})$.

$$v(\bar{X}, S) :- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)$$

Note that unlike the view v from Section 5.2.1, all of the variables that are bound outside of the *setof* subgoal in v are *not* included in the head of the v .

To facilitate our presentation of differential rules for v^+ and v^- , we define views ψ_u , ϕ_u , and v_u as follows.

$$\begin{aligned} \psi_u(\bar{X}) &:- \psi^+(\bar{X}, \bar{Y}) \\ \psi_u(\bar{X}) &:- \psi^-(\bar{X}, \bar{Y}) \\ \phi_u(\bar{X}) &:- \phi^+(\bar{X}, \bar{Z}) \\ \phi_u(\bar{X}) &:- \phi(\bar{X}, \bar{Z}), \neg\phi^-(\bar{X}, \bar{Z}) \\ v_u(\bar{X}) &:- v(\bar{X}, S) \end{aligned}$$

The following differential rules compute the differentials v^+ and v^- as views over v , ϕ , ψ , and the differentials ϕ^+ , ϕ^- , ψ^+ , and ψ^- .

$$v^-(\bar{X}, S) :- \phi^-(\bar{X}, \bar{Z}), v(\bar{X}, S), \neg\phi_u(\bar{X}) \quad (\Gamma_1)$$

$$\begin{aligned} v^+(\bar{X}, S) &:- \phi^+(\bar{X}, \bar{Z}), \neg v_u(\bar{X}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S_1), \text{setof}(\bar{Y}, \psi^+(\bar{X}, \bar{Y}), S_2), \\ &\quad \text{setof}(\bar{Y}, \psi^-(\bar{X}, \bar{Y}), S_3), S = (S_1 \setminus S_3) \cup S_2 \end{aligned} \quad (\Gamma_2)$$

$$\begin{aligned} v^+(\bar{X}, S) &:- \psi_u(\bar{X}), v(\bar{X}, S_1), \phi_u(\bar{X}), \text{setof}(\bar{Y}, \psi^+(\bar{X}, \bar{Y}), S_2), \\ &\quad \text{setof}(\bar{Y}, \psi^-(\bar{X}, \bar{Y}), S_3), S = (S_1 \setminus S_3) \cup S_2 \end{aligned} \quad (\Gamma_3)$$

$$v^-(\bar{X}, S) :- \psi^+(\bar{X}, \bar{Y}), v(\bar{X}, S) \quad (\Gamma_4)$$

$$v^-(\bar{X}, S) :- \psi^-(\bar{X}, \bar{Y}), v(\bar{X}, S) \quad (\Gamma_5)$$

To simplify our presentation of differential rules, we use the expressions $S_1 \cup S_2$ and $S_1 \setminus S_2$ to characterize union and difference of two lists S_1 and S_2 . The list functions corresponding to $\cup$ and $\setminus$ are defined in Appendix B.

Lemma 5.2.2. *The differential rules $\Gamma_1 - \Gamma_5$ correctly maintain a view containing a setof subgoal where not all of the variables that are bound outside the setof subgoal are included in the head of the view.*

Proof. Suppose the tuple $\phi(\bar{x}, \bar{y})$ is deleted from the extension of ϕ . In this scenario, an existing tuple $v(\bar{x}, s)$ may only be deleted from the extension of v if there are *no alternative derivations* of

$v(\bar{x}, s)$. This situation did not apply to $\Delta_1 - \Delta_5$ as all of the arguments of ϕ were included in the head of the rule. This update is captured by the differential rule Γ_1 .

Suppose the tuple $\phi(\bar{x}, \bar{y})$ is inserted into the extension of ϕ . If a tuple of the form $v(\bar{x}, s)$ does not exist in the extension of v , then it is inserted into v , where $s = \{\bar{y} \mid \neg\psi(\bar{x}, \bar{y}) \wedge (\psi(\bar{x}, \bar{y}) \vee \psi^+(\bar{x}, \bar{y}))\}$. This update is captured by the differential rule Γ_2 . However, if a tuple of the form $v(\bar{x}, s)$ exists in the extension of v , then it is *updated* depending on updates to ψ 's extension Γ_3 .

Suppose the extension of ϕ *does not* change. In this scenario, *no* new tuples should be inserted into or deleted from the extension of v . However, existing tuples may be *updated*. More specifically, the set argument in existing tuples may be updated based on updates to the extension of ψ . Suppose the extension of v contains the tuple $v(\bar{x}, s)$, $s_i = \{\bar{y} \mid \psi^+(\bar{x}, \bar{y})\}$, and $s_d = \{\bar{y} \mid \psi^-(\bar{x}, \bar{y})\}$. Then, as a result of the update, the tuple $v(\bar{x}, s)$ should be updated to $v(\bar{x}, s')$, where $s' = (s \setminus s_d) \cup s_i$. Updates of this form are captured by the differential rules $\Gamma_3 - \Gamma_5$. $\square$

5.3 Efficient Incremental Maintenance

In the previous section, we discussed how we could characterize the changes to a materialized view as a function of the extensions of the view, extensions of the underlying base relations, and the changes to the base relations. In this section, we analyze the *worst-case* time complexity of using the above differential rules approach, identify a performance bottleneck, and present optimizations to alleviate this bottleneck.

5.3.1 Time Complexity

We analyse the performance of differential rules $\Delta_1 - \Delta_5$ and $\Gamma_1 - \Gamma_5$ for scenarios where the underlying dataset is *indexed*. In our analysis, we consider two forms of indexing.

- **Relational Indexing.** In this form of indexing, for every base relation or materialized view, we have a list of all factoids in the dataset containing that relation.
- **Full Indexing.** In this form of indexing, for every fully or partially ground atom containing a base relation or materialized view, we have a list of all factoids in the dataset matching the specified atom.

To illustrate the differences between relational indexing and full indexing, consider a unary base relation p and a binary base relation r . Suppose the size of the dataset containing p is N and the sizes of the extensions of p and r are N_p and N_r respectively.

Under relational indexing, the number of steps required to evaluate subgoals of the form $p(X)$ where X is a ground term is N_p . In addition, the number of steps required to evaluate subgoals of the form $r(X, Y)$ where X is a ground term and Y is a variable is N_r .

Under full indexing, the number of steps required to evaluate subgoals of the form $p(X)$ where X is a ground term is $\log N_p$. In addition, the number of steps required to evaluate subgoals of the form $r(X, Y)$ where X is a ground term and Y is a variable is $\log N_r + N_r$. In the worst case, there may be N_r tuples of the form $r(X, Y)$; as a result, additional N_r steps are required.

Assumptions. To simplify our analysis we make the following assumptions:

- Subgoals are evaluated in order from left to right and the remaining subgoals are subsequently evaluated for each solution obtained. This implementation strategy, called *nested loop evaluation*, is standard in logic programming systems.
- The number of changes to the base relations are $O(1)$. As a consequence, the size of the differentials of base relations in our analysis will be denoted using the constant c .
- $O(|\psi| \times \log |\psi|)$ steps are required to evaluate a subgoal of the form $setof(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)$.

Suppose the extensions of v , ϕ , and ψ consist of n_v , n_ϕ , and n_ψ tuples respectively. The time complexity of differential rules $\Delta_1 - \Delta_5$ and $\Gamma_1 - \Gamma_5$ are summarized in Tables 5.1a and 5.1b. We presents details regarding the computation of the worst-case time complexity in Appendix A.2. We note that this analysis reflects the worst-case time complexity of differential rules for non-recursive views. This can be easily extended to recursive views by multiplying the times in Tables 5.1a and 5.1b by the depth of recursion, which is $O(n_v)$.

Indexing	Δ_1	Δ_2	Δ_3	Δ_4	Δ_5
Relational	$O(n_v)$	$O(n_\psi \times \log n_\psi)$	$O(n_v)$	$O(n_v)$	$O(n_v)$
Full	$O(\log n_v)$	$O(n_\psi \times \log n_\psi)$	$O(\log n_v)$	$O(\log n_v)$	$O(\log n_v)$

(a) Analysis of differential rules $\Delta_1 - \Delta_5$

Indexing	Γ_1	Γ_2	Γ_3	Γ_4	Γ_5
Relational	$O(n_v + n_\phi)$	$O(n_v + n_\psi \times \log n_\psi)$	$O(n_v + n_\phi)$	$O(n_v)$	$O(n_v)$
Full	$O(\log n_v + n_\phi)$	$O(\log n_v + n_\psi \times \log n_\psi)$	$O(\log n_v + n_\phi)$	$O(\log n_v)$	$O(\log n_v)$

(b) Analysis of differential rules $\Gamma_1 - \Gamma_5$

Table 5.1: Worst-case time complexity of differential rules for Datalog_A

Performance bottleneck. From the complexity analysis presented in Tables 5.1a and 5.1b, we note that the time taken by the differential rule Γ_1 is an order of magnitude higher than Δ_1 . This is because of the additional time needed to figure out whether a tuple is to be deleted from the extension of v or if there are alternative derivations of a tuple of v . We illustrate this bottleneck in the following example.

Example 5.3.1. Consider binary base relations p and r whose extensions consist of the tuples $p(a, 1)$, $p(a, 2)$, $p(b, 1)$, $r(a, 1)$, $r(a, 2)$ and $r(b, 2)$, and a materialized view v that is defined using r as follows.

$$v(X, S) :- p(X, Z), \text{ setof}(Y, r(X, Y), S)$$

The extension of v consists of the tuples $v(a, [1, 2])$ and $v(b, [2])$.

Suppose the tuple $p(a, 2)$ is deleted. In this case, do we delete the tuple $v(a, [1, 2])$, update the tuple by replacing the set argument, or leave the tuple unchanged? There are two possible derivations of $v(a, [1, 2])$. One of these derivations includes $p(a, 1)$ and the other $p(a, 2)$. As a result, the extension of v remains unchanged when $p(a, 2)$ is deleted. However, the deletion of $p(a, 2)$ affects the outcome of *future deletions* of tuples of the form $p(a, X)$.

Suppose the tuple $p(a, 1)$ is subsequently deleted. As a result of this deletion, there are *no* derivations of $v(a, [1, 2])$. Therefore, the tuple $v(a, [1, 2])$ is deleted.

5.3.2 Optimizations

The performance bottleneck discussed in Section 5.3.1 is *not* an artifact of *setof* subgoals. Rather, the additional time required is a direct consequence of the presence of variables in the body of a view's definition that are *not* included in the head of the view.

One technique to ameliorate this performance bottleneck is to *keep track of the derivations* of the tuples in the view's extension.

Consider a variant of the scenario presented in Example 5.3.1 where we maintain an auxiliary view v_c such that for every tuple $v(X, Y)$ in the extension of v , the extension of v_c contains the tuple $v_c(X, Y, C)$ where C is the number of derivations of the tuple $v(X, Y)$. Before any updates occur, the extension of v_c would consist of the tuples $v_c(a, [1, 2], 2)$ and $v_c(b, [2], 1)$. When a tuple of the form $p(X, Y)$ is deleted and *no other* tuples of the form $p(X, Z)$ are inserted, we *delete* the tuple $v(X, S)$ *if and only if* $v_c(X, S, 1)$ is in the extension of v_c . This observation is leveraged by the Counting Algorithm presented in [15] to incrementally maintain Datalog views whose tuples may have multiple derivations.

We use this idea of maintaining counts of tuple derivations to efficiently maintain Datalog_A views using differential rules in the CReaM algorithm presented below.

With regards to the CReaM algorithm, we note that:

- Steps 1–3 are applied when the view v is materialized. These steps are *not* executed when the extensions of ϕ and/or ψ are updated.
- Steps 1–3 of the CReaM algorithm need not be applied if *all* of the variables in the body of v are included in the head of v .

Algorithm 5.1 CReaM Algorithm**Inputs:**

- (a) Materialized view $v(\bar{W})$ defined as $v(\bar{W}) :- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)$
- (b) Differential relations: $\phi^+(\bar{X}, \bar{Z})$, $\phi^-(\bar{X}, \bar{Z})$, $\psi^+(\bar{X}, \bar{Y})$, and $\psi^-(\bar{X}, \bar{Y})$

Step 1: Rewrite v using an auxiliary view v_a where all of the free variables in the body of v are included in the head of v_a .

$$\begin{aligned} v(\bar{W}) &:- v_a(\bar{X}, \bar{Z}, S) \\ v_a(\bar{X}, \bar{Z}, S) &:- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S) \end{aligned}$$

Step 2: Materialize the auxiliary view v_a .

Step 3: Maintain the **count** of tuple derivations in the view v using the Counting Algorithm [15].

Step 4:

- (a) Apply the differential rules $\Delta_1 - \Delta_5$ to compute the updates to the extension of v_a .
- (b) Use Counting Algorithm to compute the insertions to (v^+) and deletions (v^-) from the extension of v .

Output: Updated instance of the view $v = (v \setminus v^-) \cup v^+$.

Theorem 5.3.1 (Correctness). *The CReaM algorithm correctly maintains materialized Datalog_A views.*

Proof. The correctness of CReaM directly follows from Lemma 5.2.1 which establishes the correctness of the differential rules $\Delta_1 - \Delta_5$ and the correctness of the Counting Algorithm [15]. $\square$

Time Complexity of CReaM. Steps 1, 2, and 3 of the CReaM algorithm are pre-processing steps that are executed ϕ or ψ are updated. Therefore, in our analysis, we need only consider the time taken to execute Step 4 of CReaM. The time complexity of Step 4(a) is characterized in Table 5.1a. Since the view v is a projection of the view v_a , the time taken to compute the differentials v^+ and v^- using the extension of v , and the differentials v_a^+ and v_a^- is $O(n_{v_a})$ or $O(\log n_{v_a})$ depending on whether the underlying dataset has relational or full indexing respectively.

Theorem 5.3.2 (Efficiency). *The speed up in time taken by CReaM to update a materialized Datalog_A view v , whose extension is of size $O(n)$, is $O(1)$ or $O(\frac{n}{\log n})$ depending on whether the underlying dataset has relational or full indexing.*

Proof. The above result follows directly by diving the columns in Table 5.1b by the corresponding columns in Table 5.1a. $\square$

5.4 Conclusions

We have extended the differential rules presented in [34] to incrementally maintain Datalog_A views containing *setof* subgoals in response to changes to the base relations. For materialized views where *all* of the variables in the body of a view's definition are included in the head of the view, we have

shown that the view is *self-maintainable* with respect to changes to the base relations in the body of the *setof* subgoal. We have also described how our proposed differential rules can be made efficient by maintaining counts of derivations of tuples in the view's extension.

Chapter 6

Query Folding

6.1 Introduction

The query folding problem was first introduced in [35] and refers to the problem of determining whether and how a supplied query over a database can be answered using views. Formally, the query folding problem is defined as follows.

Definition 6.1.1 (Query folding). Given a query q and views $v_1, v_2, \dots, v_n$, is there a query q' that can be defined using $v_1, v_2, \dots, v_n$ such that q and q' are equivalent? If not, is there a query q' that can be defined using $v_1, v_2, \dots, v_n$ such that q' is maximally contained in q ?

In Definition 6.1.1, the rewriting q' is commonly referred to as a *query plan* for query q . Techniques for folding queries can be applied to optimize database queries as they can potentially reuse pre-computed answers. In these scenarios it is essential that the generated query plans are equivalent to the supplied queries. Query folding techniques may also be used in a Local-as-View (LAV) data integration [17] to answer queries. In such scenarios, it suffices to generate query plans that are maximally contained in the query.

A comprehensive survey of query folding techniques for Datalog is presented in [17]. While the problem of folding conjunctive Datalog queries using conjunctive views has received significant attention, only a small fraction of this work addresses cases where (a) the queries are recursive [8] or (b) the views and/or the query contain aggregates [1, 6, 14, 37]. The folding techniques for case (b) provide support for built-in aggregates only, e.g., *min*, *max*, *sum* or *count*, and assume that the aggregation operators used in query and the views are *identical*.

In this chapter, we present a novel approach for folding aggregate queries. In contrast to prior folding techniques that deal with aggregate queries on a case-by-case basis depending on the aggregation function and the underlying semantics e.g. bag or set, our approach focuses on folding the generalized aggregate operator *setof*. To do so, we extend the Inverse Method that was introduced

in [8] to fold Datalog $\neg$ queries using conjunctive views.

6.2 Inverse Method for Datalog without Aggregates

The Inverse Method presented in [8] generates rewritings of Datalog queries by *inverting* view definitions. Consider a Datalog query Q and a set of view definitions V . To answer Q using the views in V , the view definitions are *inverted*.

Definition 6.2.1 (Inverse rules). The inverse rules (v^{-1}) of a view v that is defined as $v(\bar{X}) :- p_1(\bar{X}_1), p_2(\bar{X}_2), \dots, p_n(\bar{X}_n)$ are the rules:

$$\begin{aligned} p_1(\bar{X}_1') &:- v(\bar{X}) \\ p_2(\bar{X}_2') &:- v(\bar{X}) \\ &\dots \\ p_n(\bar{X}_n') &:- v(\bar{X}) \end{aligned}$$

For all $1 \leq i \leq n$, $\bar{X}_i'$ is obtained from $\bar{X}_i$ by replacing variables that *do not* appear in $\bar{X}$ with Skolem terms $f_{i,j}(\bar{X})$.

The inverse rules for a set of view definitions V is denoted as V^{-1} and is given by $\bigcup_{v \in V} v^{-1}$. Given a query Q and a set of view definitions V , the output of the Inverse Method is the query plan $Q \cup V^{-1}$. Note that this query plan may contain Skolem terms. However, these Skolem function symbols can be eliminated from the generated query plan by applying the transformation described in [8].

We illustrate the application of the Inverse Method using the following example.

Example 6.2.1. Consider a graph whose edges are represented using a binary base relation called *edge*, and a query q and view v that are defined over *edge* as follows.

$$\begin{aligned} q(X, Y) &:- \text{edge}(X, Y) \\ q(X, Z) &:- \text{edge}(X, Y), q(Y, Z) \\ v(X, Z) &:- \text{edge}(X, Y), \text{edge}(Y, Z) \end{aligned}$$

Query q computes the transitive closure of the *edge* relation. Note that any *folding* of q using v alone *cannot* compute all answers to q . This is because the view v only stores pairs of nodes in the graph that are connected by a path of length 2. The best one can hope to achieve is to compute all paths of *even* length.

Applying the Inverse Method on q and v results in the following query plan which contains the

inverse rules v^{-1} .

$$\begin{aligned}
 q(X, Y) &:- \text{edge}(X, Y) \\
 q(X, Z) &:- \text{edge}(X, Y), q(Y, Z) \\
 \text{edge}(X, f(X, Z)) &:- v(X, Z) \\
 \text{edge}(f(X, Z), Z) &:- v(X, Z)
 \end{aligned}$$

Suppose the base relation edge represents the graph shown in Figure 6.1.



Figure 6.1: Graph represented by the edge relation

In this case, extension of v consists of the tuples $v(a, c)$, $v(b, d)$, $v(c, e)$. Evaluation of the query plan generated using the Inverse Method on the extension of v results in the answers: $q(a, c)$, $q(b, d)$, $q(c, e)$, $q(a, e)$, which corresponds to paths of even length in the graph.

Theorem 6.2.1. *Given a Datalog query Q and a set of conjunctive view definitions V , the query plan $Q \cup V^{-1}$ is maximally contained in Q .*

Proof. This result was shown in [8]. □

6.3 Inverting Views containing *setof* subgoals

In the remainder of this chapter, we'll extend the Inverse Method to fold Datalog_A queries using *conjunctive aggregate views*.

A conjunctive aggregate view has the general form $v(\bar{X}) :- s_1(\bar{X}_1), s_2(\bar{X}_2), \dots, s_n(\bar{X}_n)$ where each s_i is either a base relation, a *setof* subgoal over base relations, or a list function.

To simplify our presentation, we first discuss the folding of Datalog_A queries using a class of conjunctive aggregate views that do *not* contain list functions. Later, in Section 6.5, we will remove this restriction.

A conjunctive *setof* view (CSQ view) is a conjunctive aggregate view where list functions do not appear in the body of the view's definition. For example, if p is a unary base relation and r is a binary base relation, then the following views are CSQ views.

$$\begin{aligned}
 v_1(X, S) &:- p(X), \text{setof}(Y, r(X, Y), S) \\
 v_2(X, S) &:- p(X), \text{setof}(Y!T, r(X, Y), \text{setof}(Z, s(Y, Z), T), S) \\
 v_3(S, V) &:- p(X), \text{setof}(Y!T, r(X, Y), \text{setof}(Z, s(Y, Z), T), S), \text{setof}(W, r(W, X), V)
 \end{aligned}$$

In this section, we'll first discuss the inversion of CSQ views that contain one, unnested *setof* subgoal. Then, we'll cover the inversion of views containing additional, potentially nested *setof* subgoals in our query folding algorithm, $\text{Inverse}_{\text{Agg}}$.

A CSQ view v containing one *setof* subgoal has one of the following forms.

- **Case 1.** $v(\bar{X}, S) :- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}, \bar{Z}), S)$. In this case, the output argument of *setof* is included in the head of the view.
- **Case 2.** $v(\bar{X}) :- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}, \bar{Z}), S)$. In this case, the output argument of *setof* is *not* included in the head of the view.

In the above view definitions, ϕ and ψ are conjunctions of base relation atoms.

6.3.1 Inverting Views that include Output of *setof*

Consider a view v that is defined as follows.

$$v(\bar{X}, S) :- \phi(\bar{X}, \bar{Z}), \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}, \bar{Z}), S)$$

Assume that ψ does *not* contain additional *setof* subgoals. Let $\phi(\bar{X}, \bar{Z})$ be the conjunction of subgoals of the form $p_i(\bar{X}_{i,1}, \bar{Z}_{i,1})$ for $1 \leq i \leq k$, and $\psi(\bar{X}, \bar{Y}, \bar{Z})$ be a conjunction of subgoals of the form $r_i(\bar{X}_{i,2}, \bar{Y}_{i,2}, \bar{Z}_{i,2})$ for $1 \leq i \leq m$.

Definition 6.3.1. Given a view v whose definition does not contain nested *setof* subgoals and includes the output argument of *setof* subgoals in the head, the inverse rules for v , denoted as v^{-1} , are defined as follows.

$$\begin{aligned} p(\bar{X}'_{1,1}, \bar{Z}'_{1,1}) &:- v(\bar{X}, S) \\ \dots & \\ p(\bar{X}'_{k,1}, \bar{Z}'_{k,1}) &:- v(\bar{X}, S) \\ r(\bar{X}'_{1,2}, \bar{Y}'_{1,2}, \bar{Z}'_{1,2}) &:- v(\bar{X}, S), \text{member}(\bar{Y}, S) \\ \dots & \\ r(\bar{X}'_{m,2}, \bar{Y}'_{m,2}, \bar{Z}'_{m,2}) &:- v(\bar{X}, S), \text{member}(\bar{Y}, S) \end{aligned}$$

The sequences of terms $\bar{X}'_{i,j}$, $\bar{Y}'_{i,j}$, and $\bar{Z}'_{i,j}$ in v^{-1} are obtained as follows.

- For all $1 \leq i \leq \max\{m, k\}$ and $j \in \{1, 2\}$, $\bar{X}'_{i,j}$ is obtained from $\bar{X}$ and S by replacing variables that are *not* S and *do not* appear in $\bar{X}$ with Skolem terms $f_{i,j,k}(\bar{X}, S)$.
- For all $1 \leq i \leq \max\{m, k\}$ and $j \in \{1, 2\}$, $\bar{Z}'_{i,j}$ is obtained from $\bar{X}$ and S using Skolem terms $g_{i,j,k}(\bar{X}, S)$.

- For all $1 \leq i \leq m$, $\bar{Y}'_{i,2}$ is obtained from $\bar{X}$ and S by replacing variables that are *not* in S and *do not* appear in either $\bar{X}$ or $\bar{Y}$ with Skolem terms $h_{i,j,k}(\bar{X}, S)$.

We illustrate inversion of views containing a *setof* subgoal in the following example.

Example 6.3.1. Consider a view v that is defined over binary base relation p and r as follows.

$$v(X, S) :- p(X, Z), \text{setof}(Y, r(X, Y), S)$$

Suppose we would like to use v to fold a query q that is defined as follows.

$$q(X, Y) :- r(X, Y), p(Y, Z)$$

Inverting v using the approach discussed above results in the following set of inverse rules (v^{-1}).

$$\begin{aligned} p(X, f(X, S)) &:- v(X, S) \\ r(X, Y) &:- v(X, S), \text{member}(Y, S) \end{aligned}$$

Suppose an extension of v consists of the tuples $v(a, [b, c])$, $v(b, [])$, and $v(c, [a, c])$. Evaluation of the query plan $\{q\} \cup v^{-1}$ on the extension of v results in the answers: $q(a, c)$, $q(c, a)$, and $q(c, c)$. Note that in this case, the query plan $\{q\} \cup v^{-1}$ is *equivalent* to q .

Dealing with nested *setof* subgoals in views. So far, we have assumed that there are no *setof* subgoals in $\psi(\bar{X}, \bar{Y}, \bar{Z})$. To eliminate this assumption, we first transform a supplied view using the technique in Appendix 1.1. This ensures that each rule in the transformed view contains at most one unnested *setof* subgoal. In a subsequent step, we invert the transformed rules.

Algorithm 6.1 Inverting views when the output of *setof* subgoals is *not* projected out

Input: View definition $v(\bar{X}_1, S) :- \phi(\bar{X}_1, \bar{X}_2), \text{setof}(\bar{Y}, \psi(\bar{X}_1, \bar{X}_2, \bar{Y}), S)$.

if $\psi(\bar{X}_1, \bar{X}_2, \bar{Y})$ contains *setof* subgoals **then**

$V_r \leftarrow$ rewriting of v using the transformation described in Appendix 1.1.

else

$V_r \leftarrow \{v(\bar{X}_1, S) :- \phi(\bar{X}_1, \bar{X}_2), \text{setof}(\bar{Y}, \psi(\bar{X}_1, \bar{X}_2, \bar{Y}), S)\}$.

end if

Let u^{-1} denote the inverse rules that are generated by applying Definition 6.3.1 to u , for each $u \in V_r$.

Output: $\bigcup_{u \in V_r} u^{-1}$

6.3.2 Inverting Views that do not include Output of *setof*

In Section 6.3, we discussed inversion of views whose definition includes the output of *setof* subgoals in the head. In this section, we will discuss inversion of views that *do not* satisfy this condition.

Suppose a view v is defined over binary base relations p and r as follows.

$$v(X) :- p(X, S), \text{setof}(Y, r(X, Y), S)$$

Clearly, view v *does not* satisfy the conditions under which Algorithm 6.1 is applicable. Let us see whether we can use the query plan generated using Algorithm 6.1 can be used to fold the following query using v .

$$q(X) :- p(X, S), \text{setof}(Y, r(X, Y), S)$$

Applying Algorithm 6.1 generates the following inverse rules, v^{-1} .

$$\begin{aligned} p(X, f(X)) &:- v(X) \\ r(X, Y) &:- v(X), \text{member}(Y, f(X)) \end{aligned}$$

The expansion of the query plan $\{q\} \cup v^{-1}$ is shown below.

$$q(X) :- v(X), \text{setof}(Y, v(X), \text{member}(Y, f(X))), f(X))$$

Since the definitions of q and v are *identical*, it should be the case that a tuple $q(x)$ is an answer to query q *iff* $v(x)$ is in the extension of view v . In the expansion of the query plan for q , the equivalence between $q(x)$ and $v(x)$ holds *iff* $f(x) = \{Y \text{ s.t. } v(X), \text{member}(Y, f(X))\}$.

To account for equivalences of this sort, we characterize properties of the *setof* operator using the following dependencies Γ_s .

$$\begin{aligned} \Gamma_{s_1} : \forall \bar{X} (\phi(\bar{X}) \wedge \text{setof}(\bar{Y}, \phi(\bar{X}), \psi(\bar{X}, \bar{Y}), S) &\iff \phi(\bar{X}) \wedge \text{setof}(\bar{Y}, \psi(\bar{X}, \bar{Y}), S)) \\ \Gamma_{s_2} : \forall S, T (\text{setof}(\bar{Y}, \text{member}(Y, S), T) &\iff S = T) \end{aligned}$$

The expansion of $\{q\} \cup v^{-1}$ is simplified by applying Γ_{s_1} and Γ_{s_2} in succession. Applying Γ_{s_1} to $v(X), \text{setof}(Y, v(X), \text{member}(Y, f(X))), f(X))$ results in $v(X), \text{setof}(Y, \text{member}(Y, f(X)), f(X))$. Applying Γ_{s_2} to the expansion in a subsequent step results in $q(X) :- v(X), f(X) = f(X)$, which can be further simplified as follows.

$$q(X) :- v(X)$$

Note that if the output S of the *setof* subgoal was a ground list, e.g., $[a, b]$, then the Skolem term

$f(X)$ would not have appeared in the inverse rules. This case would be analogous to Case 1, when the output argument of the *setof* subgoal is included in the head of the view's definition.

Query folding algorithm. In the following, we present a query folding algorithm, called $\text{Inverse}_{\text{Agg}}$, to fold Datalog_A queries using CSQ views. In general, CSQ view definitions may contain multiple, potentially nested *setof* subgoals. To fold Datalog_A queries using such views, $\text{Inverse}_{\text{Agg}}$ algorithm first transforms the supplied views using the procedure described in Appendix 1.2. This ensures that each rule in the transformed view definitions contains exactly one *setof* subgoal. Subsequently, Algorithm 6.1 and dependencies Γ_s are applied to generate a query plan.

Algorithm 6.2 ($\text{Inverse}_{\text{Agg}}$) Generating query plans for Datalog_A queries using CSQ views

Input: Query Datalog_A query q and views $v_1, v_2, \dots, v_n$.

Let $V_r \leftarrow$ rewriting of $v_1, v_2, \dots, v_n$ using the transformation described in Appendix 1.2.

Let v^{-1} denote the inverse rules that are generated by applying Algorithm 6.1 to v , for $v \in V_r$.

$V^{-1} \leftarrow \bigcup_{v \in V_r} v^{-1}$

Output: $\{q\} \cup V^{-1} \cup \Gamma_s$

6.4 Maximally Contained Query Plans

We start off this section with a negative result. The $\text{Inverse}_{\text{Agg}}$ algorithm *does not* generate maximally contained query plans for Datalog_A queries in general. In fact, the query plan *may not* even be contained in the supplied query. We present an example of a query below for which the $\text{Inverse}_{\text{Agg}}$ algorithm results in a query plan that is not contained in the supplied query.

Example 6.4.1 (Non-contained query plan). Consider a database that consists of a unary base relation p and a binary base relation r . Consider a query q and a view v that are defined over p and r as follows.

$$\begin{aligned} q(a) &:- \text{setof}(Y, r(a, Y), []) \\ v(X, S) &:- p(X), \text{setof}(Y, r(X, Y), S) \end{aligned}$$

Suppose the extensions of p and r consist of the tuples $p(b)$ and $r(a, 1)$ respectively. In this case, the extension of v consists of tuple $v(b, [])$. The query plan that is generated by $\text{Inverse}_{\text{Agg}}$ algorithm

is as follows.

$$\begin{aligned} q(a) &:- \text{setof}(Y, r(a, Y), []) \\ p(X) &:- v(X, S) \\ r(X, Y) &:- v(X, S), \text{member}(Y, S) \end{aligned}$$

Evaluating the query plan results in the answer $q(a)$, which is *not* obtained by evaluating the query q on the extensions of p and r .

Expressing negation through *setof*. To simplify our presentation, we assume that negated subgoals that appear in the query and view definitions are replaced using *setof* subgoals using the transformation described in Lemma 4.1.1.

Information loss in inverse rules. Note that the non-containment of the query plan generated by the $\text{Inverse}_{\text{Agg}}$ algorithm in Example 6.4.1 is due to the incomplete information about the extension of r being captured in the inverse rule $r(X, Y) :- v(X, S), \text{member}(Y, S)$. This inverse rule characterizes that a tuple $r(x, y)$ is in the extension of r if there is some binding of S such that $v(x, S)$ is in the extension of v and $y \in S$. That said, the inverse rule fails to characterize that there may be other tuples of the form $r(X, Y)$ in the extension of r even where there does not exist a binding of S such that $v(X, S)$ is in the extension of v . This non-containment result for $\text{Inverse}_{\text{Agg}}$ algorithm is not surprising since it was shown in [3] that Inverse Method [8] does not generate contained query plans for Datalog $^-$ queries.

However, all is not lost. In the following, we characterize a class of query folding problems for which the $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans, and discuss the applicable restrictions below.

6.4.1 Folding a Restricted Class of Queries

Let us revisit Example 6.4.1. Suppose we redefine query q as follows.

$$q(a) :- \text{setof}(Y, r(a, Y), H!T)$$

Consider the folding of q using v . In this case, the $\text{Inverse}_{\text{Agg}}$ algorithm generates the following query plan $\{q\} \cup v^{-1}$.

$$\begin{aligned} q(a) &:- \text{setof}(Y, r(a, Y), H!T) \\ p(X) &:- v(X, S) \\ r(X, Y) &:- v(X, S), \text{member}(Y, S) \end{aligned}$$

By contrast to the query plan from Example 6.4.1, the above query plan is contained in q . The containment directly follows from the fact that there exists a binding of S such that $v(a, S)$ is in the extension of $v \iff$ there exists a binding of Y such that $r(X, Y)$ is in the extension of $Y \iff q(a)$ is an answer of the query q .

The difference in the containment relationship between the generated query plan and the query from Example 6.4.1 can be accounted for as follows.

In Example 6.4.1, $q(a)$ is an answer to q if there are *no* bindings of Y such that $r(a, Y)$ is in the extension of r . Unfortunately, there is no equivalent rewriting of $r(a, Y)$ using the view v . At best, the extension of r that is computed using the inverse rules v^{-1} are maximally contained in r . As a result, there is no way to guarantee that the set of all bindings Y such that $r(a, Y)$ is in the extension of v is empty.

By contrast, for the query q defined above, $q(a)$ is an answer to q if there is *some* binding of Y such that $r(a, Y)$ is in the extension of r . This implies that, if there exists a binding of the set S such that $v(a, S)$ is in the extension of v , then $q(a)$ is an answer to q . We can characterize queries of this sort using the notion of *monotonicity* defined below.

Definition 6.4.1 (Monotonic Datalog_A rule). A Datalog_A rule of the form $q(\bar{H}) :- s_1, s_2, \dots, s_n$, where $q(\bar{W})$ denotes the head of the rule, and $s_1, s_2, \dots, s_n$ denote the subgoals, is monotonic *iff* all of the following conditions hold.

- For all $1 \leq i \leq n$ such that s_i is of the form $setof(\bar{X}, \phi(\bar{X}, \bar{Y}), S)$ where $\phi(\bar{X}, \bar{Y})$ is a conjunction of subgoals and for every dataset D , addition of new tuples to the extension of ϕ , denoted as ϕ^+ , *cannot* result in the deletion of an answer from the extension of q , i.e., the differential q^- is empty.
- For all $1 \leq i \leq n$, if s_i is the form $r(\bar{X}, S, \bar{Y})$ where S appears in the output argument of a *setof* subgoal, then $\forall S, T, \bar{A}, \bar{B} (r(\bar{X}, S, \bar{Y}) \wedge S \subseteq T \implies r(\bar{X}, T, \bar{Y}))$

Definition 6.4.2 (Monotonic Datalog_A query). A Datalog_A query q is monotonic *iff* every rule that appears in the definition of q is monotonic.

The query q from Example 6.4.1 is not monotonic. This is because an addition of a tuple to the extension of r , characterized as $r^+(a, X)$, may result in the deletion of the tuple $q(a)$ from the extension of q . By contrast, the query q that is defined as $q(a) :- setof(Y, r(a, Y), H!T)$ is monotonic.

For monotonic queries, our Inverse_{Agg} algorithm generates maximally contained query plans.

Lemma 6.4.1. A Datalog_A rule of the form $q(\bar{H}) :- s_1, s_2, \dots, s_n$ is not monotonic if there exists $1 \leq i \leq n$, such that s_i is of the form $setof(\bar{X}, \phi(\bar{X}, \bar{Y}), S)$, S is a variable, and $\bar{H}$ includes S .

Proof. Suppose that for some k , $1 \leq k \leq n$, s_k is of the form $setof(\bar{X}, \phi(\bar{X}, \bar{Y}), S)$ and $\bar{H}$ includes S .

Let us assume that $q(\bar{H}) :- s_1, s_2, \dots, s_n$ is monotonic. Using the differential rules Δ_4 – Δ_5 and Γ_4 – Γ_5 and in Chapter 5 we can show that additions to ϕ , denoted using the differential ϕ^+ , result in a non-empty differential q^- . Therefore, q is *not* monotonic. $\square$

Theorem 6.4.1. *The $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans for monotonic queries.*

Proof. Without loss of generality we assume that every rule in the supplied monotonic query contains at most one *setof* subgoal. In addition, we assume that the *setof* subgoals are not nested. For rules that do not satisfy these conditions, we apply the transformations described in Appendix 1.1 and 1.2.

We prove our result using induction of the number of strata in the transformed query. In the base case, $n = 0$ and every rule is of the general form: $q(\bar{X}) :- \phi(\bar{X}, \bar{Y})$. Let us consider the query plan that is generated by the $\text{Inverse}_{\text{Agg}}$ algorithm. Let V'_r be the set of views $\{v \text{ s.t. } v \in V_r \wedge v \text{ does not contain a setof subgoal}\} \cup \bigcup_{v \in V} S_v$ if v contains a *setof* subgoal, where S_v is defined as follows.

- If v has the general form $v(\bar{X}, S) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$, then S_v consists of the rules $v_\phi(\bar{X}) :- \phi(\bar{X}, \bar{Y})$ and $v_\psi(\bar{X}, \bar{Z}) :- \phi(\bar{X}, \bar{Y}), \psi(\bar{X}, \bar{Z})$.
- If v has the general form $v(\bar{X}) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$ and S is a ground list, then S_v consists of the rules $v_\phi(\bar{X}) :- \phi(\bar{X}, \bar{Y})$ and $v_\psi(\bar{X}) :- \phi(\bar{X}, \bar{Y}), \psi(\bar{X}, \bar{z})$ for every $\bar{z} \in S$.
- If v has the general form $v(\bar{X}) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$, then S_v consists of the rules $v_\phi(\bar{X}) :- \phi(\bar{X}, \bar{Y})$ and $v_\psi(\bar{X}) :- \phi(\bar{X}, \bar{Y}), \psi(\bar{X}, \bar{Z})$.

By definition, $q(\bar{x})$ is an answer obtained by the query plan generated by the $\text{Inverse}_{\text{Agg}}$ algorithm with query q and views V_r as inputs *iff* $q(\bar{x})$ is an answer obtained by the query plan generated by the $\text{Inverse}_{\text{Agg}}$ algorithm with query q and views V'_r as inputs. Since V'_r is a set of conjunctive views, it follows from the minimal containment result in [8] that the query plan generated by the $\text{Inverse}_{\text{Agg}}$ algorithm is maximally contained in q .

Since V'_r is a set of conjunctive views and q is a conjunctive query, the $\text{Inverse}_{\text{Agg}}$ algorithm generates a maximally contained query plan using V'_r .

Let us assume that the $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans for monotonic queries with strata $< n$. To finish our proof, it suffices to show that the $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans for monotonic queries with strata $= n$.

Consider a monotonic query with strata $= n$ and let V^{-1} denote the inverse rules generated by the $\text{Inverse}_{\text{Agg}}$ algorithm. The rules of the query have the following general form.

$$q(\bar{W}) :- \phi(\bar{X}, \bar{Y}), \tau(\bar{U}, S), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$$

Also, q is monotonic. From Definition 6.4.1 it follows that S does not appear in $\bar{W}$ and $\forall S, T, \bar{U} (S \subseteq T \wedge \tau(\bar{U}, S) \implies \tau(\bar{U}, T))$.

First, we prove that the query plan $\{q\} \cup V^{-1}$ is contained in q . Consider queries q_1 , q_2 , and q_3 that are defined as follows.

$$\begin{aligned} q_1(\bar{X}, \bar{Y}) &:- \phi(\bar{X}, \bar{Y}) \\ q_2(\bar{U}, S) &:- \tau(\bar{U}, S) \\ q_3(\bar{X}, \bar{Z}) &:- \psi(\bar{X}, \bar{Z}) \end{aligned}$$

By construction, the number of strata in q_1 , q_2 , and q_3 is $< n$. Therefore, the query plans $\{q_i\} \cup V^{-1}$ are maximally contained in q_i for $1 \leq i \leq 3$. Therefore, $\{q\} \cup V^{-1}$ is contained in q .

To prove that $\{q\} \cup V^{-1}$ is maximally contained in q , we assume that there exists a different query plan which is maximally contained in q . Let $q(\bar{w})$ be an answer that is generated by this plan, but not by $\{q\} \cup V^{-1}$. Let $\phi(\bar{x}, \bar{y})$, $\tau(\bar{u}, s)$, and $setof(\bar{Z}, \psi(\bar{x}, \bar{Z}), s)$ be nodes in the derivation tree of $q(\bar{w})$. Maximal containment of the query plans $\{q_i\} \cup V^{-1}$ in q_i for $1 \leq i \leq 3 \implies \phi(\bar{x}, \bar{y})$, $\tau(\bar{x}, s)$ and $setof(\bar{Z}, \psi(\bar{x}, \bar{Z}), t)$ are also generated by $\{q\} \cup V^{-1}$, where $s \subseteq t$. Since q is monotonic, it follows that the tuple $\tau(\bar{x}, s)$ is also generated by $\{q\} \cup V^{-1}$. As a result, $q(\bar{w})$ is generated by $\{q\} \cup V^{-1}$. Therefore, $\{q\} \cup V^{-1}$ is maximally contained in q . $\square$

6.4.2 Foldings that use a Restricted Class of Views

So far, we have discussed the generation of maximally contained query plans by restricting the type of queries. In the following, we show that $Inverse_{Agg}$ algorithm generates maximally contained query plans for *all* Datalog_A queries if we restrict the view definitions that are supplied as inputs to the algorithm.

Definition 6.4.3 (Canonical aggregate views). The canonical aggregate view v of a n -ary relation r is either defined as $v(X_1, X_2, \dots, X_n) :- r(X_1, X_2, \dots, X_n)$, or using the following rule.

$$v(X_{i_1}, X_{i_2}, \dots, X_{i_k}, S) :- r(X_1, X_2, \dots, X_n), setof(\bar{Y}, r(\bar{Z}), S)$$

In the above rule, $i_1 < i_2 < \dots < i_k$ and $1 \leq i_j \leq n$ for all $1 \leq j < n$, and $\bar{Y}$ and $\bar{Z}$ denote sequences of variables $Y_1, Y_2, \dots, Y_{n-k}$ and $Z_1, Z_2, \dots, Z_n$, such that for $Z_j = X_j$ for all $j \in \{i_1, i_2, \dots, i_k\}$, and $Z_j = Y_j$ for all $j \notin \{i_1, i_2, \dots, i_k\}$.

There are 2^n different canonical views of a n -ary relation.

Example 6.4.2. Consider a binary relation r . The different canonical views of r are as follows.

$$\begin{aligned} v_1(S) &:- r(X_1, X_2), \text{ setof}(Y_1!Y_2, r(Y_1, Y_2), S) \\ v_2(X_1, S) &:- r(X_1, X_2), \text{ setof}(Y_2, r(X_1, Y_2), S) \\ v_3(X_2, S) &:- r(X_1, X_2), \text{ setof}(Y_1, r(Y_1, X_2), S) \\ v_4(X_1, X_2) &:- r(X_1, X_2) \end{aligned}$$

Lemma 6.4.2. *Given a query q of the form $q(\bar{X}) :- r(\bar{X})$ and a canonical aggregate view of the relation r , an equivalent query plan is generated by the $\text{Inverse}_{\text{Agg}}$ algorithm.*

Proof. It suffices to prove our result only for cases where the supplied canonical view contains a *setof* subgoal. Suppose a canonical view of r is of the form: $v(\bar{X}, S) :- r(\bar{X}, \bar{Y}), \text{ setof}(\bar{Z}, r(\bar{X}, \bar{Z}), S)$. A tuple $v(\bar{x}, s)$ is in the extension of v iff there exists a tuple of the form $r(\bar{x}, \bar{Y})$ in the extension of r and $s = \{\bar{Z} \text{ s.t. } r(\bar{x}, \bar{Z})\}$.

Inverting the definition of v results in the following inverse rule.

$$r(X, \bar{Y}) :- v(\bar{X}, S), \text{ member}(\bar{Y}, S)$$

Assume that the query plan generated by $\text{Inverse}_{\text{Agg}}$ algorithm is not equivalent to q . It must be the case that there exists a tuple $q(x, y)$ that is not obtained by evaluating the query plan of the extension of v . Since $q(x, y)$ is in the extension of q , it must be the case that $r(x, y)$ is in the extension of r and there exists a tuple of the form $v(x, s)$ in the extension of v such that $y \in s$. Therefore, the query plan generated by $\text{Inverse}_{\text{Agg}}$ algorithm is equivalent to q . $\square$

In the following, we show that the $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans if the canonical aggregate views of relations that appears in the body of a *setof* subgoal in the query are included as inputs of the query folding problem.

Theorem 6.4.2. *The $\text{Inverse}_{\text{Agg}}$ algorithm generates maximally contained query plans for Datalog_A queries if, for every relation r that appears in a negated subgoal or in the body of a *setof* subgoal, a canonical aggregate view of r is included as a view.*

Proof. The structure of this proof is similar to the proof of Theorem 6.4.1 and we prove our result using induction on the number of strata in query that is transformed using the procedures described in Appendix 1.1 and 1.2.

In the base case, the number of strata is 0 $\implies$ there are no *setof* subgoals in the query. The proof of this base case is identical to the proof of the base case in Theorem 6.4.1.

Let us assume that our result holds if the number of strata in the transformed query $< n$.

Consider a transformed query q with n strata. The rules for q have the following general form.

$$q(\bar{W}) :- \phi(\bar{X}, \bar{Y}), \tau(\bar{U}, S), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$$

Note that in this case, since q is not required to be monotonic, S may appear in $\bar{W}$.

Let $\{q\} \cup V^{-1}$ denote the query plan that is generated by $\text{Inverse}_{\text{Agg}}$ algorithm. First, we show that $\{q\} \cup V^{-1}$ is contained in q . In a subsequent step, we prove that $\{q\} \cup V^{-1}$ is maximally contained in q .

Consider queries q_1 , q_2 , and q_3 that are defined as follows.

$$\begin{aligned} q_1(\bar{X}, \bar{Y}) &:- \phi(\bar{X}, \bar{Y}) \\ q_2(\bar{U}, S) &:- \tau(\bar{U}, S) \\ q_3(\bar{X}, \bar{Z}) &:- \psi(\bar{X}, \bar{Z}) \end{aligned}$$

By construction, the number of strata in q_1 , q_2 , and q_3 is $< n$. Therefore, the query plans $\{q_i\} \cup V^{-1}$ are maximally contained in q_i for $1 \leq i \leq 3$.

For every relation r that appears in ψ , a canonical aggregate view of r is included in the views that are supplied as input to the $\text{Inverse}_{\text{Agg}}$ algorithm. From Lemma 6.4.2 it follows that the query plan $\{q_3\} \cup V^{-1}$ is, in fact, equivalent to q_3 . Therefore, $\{q\} \cup V^{-1}$ is contained in q .

To prove that $\{q\} \cup V^{-1}$ is maximally contained in q , we assume that there exists a different query plan which is maximally contained in q . Let $q(\bar{w})$ be an answer that is generated by this plan, but not by $\{q\} \cup V^{-1}$. Let $\phi(\bar{x}, \bar{y})$, $\tau(\bar{u}, s)$, and $\text{setof}(\bar{Z}, \psi(\bar{x}, \bar{Z}), s)$ be nodes in the derivation tree of $q(\bar{w})$. Maximal containment of the query plans $\{q_i\} \cup V^{-1}$ in q_i for $1 \leq i \leq 2$ and equivalence of the query plan $\{q_3\} \cup V^{-1}$ with q_3 implies that $q(\bar{w})$ is generated by $\{q\} \cup V^{-1}$ as well. Therefore, $\{q\} \cup V^{-1}$ is maximally contained in q . $\square$

6.5 Inverting Views with List Functions and Improvements

One of the limitations of the Inverse Method [8] and of our $\text{Inverse}_{\text{Agg}}$ algorithm is that the view definitions cannot themselves be recursive. As a consequence, the $\text{Inverse}_{\text{Agg}}$ algorithm cannot be used to fold Datalog_A queries when the view definitions contain arbitrary list functions.

We illustrate the problem with inverting views with arbitrary list functions in the example below.

Example 6.5.1. Consider a view v that is defined over a unary base relation p and a binary base

relation r using the list function sum as follows.

$$\begin{aligned} v(X, A) &:- p(X), setof(Y, r(X, Y), S), sum(X, A) \\ &sum([], 0) \\ sum(X!Y, S) &:- sum(Y, A), S = X + A \end{aligned}$$

Inverting the definition of v results in the following inverse rule.

$$\begin{aligned} p(X) &:- v(X, A) \\ r(X, Y) &:- v(X, A), member(Y, f(X, A)) \\ sum(f(X, A), A) &:- v(X, A) \end{aligned}$$

If the extension of v is not empty, then the bottom-up evaluation of the inverse rule, coupled with the recursive definition of sum , will never terminate and will generate the tuples: $sum([], 0)$, $sum(f(X, A), A)$, $sum([f(X, A), f(X, A)], 2 \times A)$, $\dots$ and so on.

From Example 6.5.1 it is evident that even for the restricted class of query folding problems for which $Inverse_{Agg}$ algorithm generates maximally contained plans, addition of arbitrary list functions to views results in non-terminating query plans. That said, in the following, we show that the $Inverse_{Agg}$ algorithm can potentially be extended to fold aggregate queries where the list functions in the query are defined as conjunctive views of the list functions that appear in the views.

Example 6.5.2. Consider a query q that is defined over a unary base relation p and a binary base relation r using the list function avg (see Appendix B) as follows.

$$q(X, A) :- p(X), setof(Y, r(X, Y), S), avg(X, A)$$

Consider the folding of q using views v_1 and v_2 that are defined as follows.

$$\begin{aligned} v_1(X, T) &:- p(X), setof(Y, r(X, Y), S), sum(X, T) \\ v_2(X, C) &:- p(X), setof(Y, r(X, Y), S), length(X, C) \end{aligned}$$

In this case, list function agg is defined as a view over the list functions sum and $length$.

$$avg(X, A) :- sum(X, S), length(X, C), A = \frac{S}{C}$$

As a result, there is an equivalent folding of q using views v_1 and v_2 .

$$q(X, A) :- v_1(X, T), v_2(X, C), A = \frac{T}{C}$$

Need to capture functional dependencies. Consider the expansion of the list function *agg* in query *q* from Example 6.5.2.

$$q_1(X, A) :- p(X), \text{setof}(Y, r(X, Y), S), \text{sum}(S, T), \text{length}(S, C), A = \frac{T}{C}$$

Consider a query plan which includes the expanded query q' , the dependencies Γ_s , and the following inverse rules (V^{-1}).

$$\begin{aligned} p(X) &:- v_1(X, T) \\ r(X, Y) &:- v_1(X, T), \text{member}(X, f(X, T)) \\ \text{sum}(f(X, T), T) &:- v_1(X, T) \\ p(X) &:- v_2(X, C) \\ r(X, Y) &:- v_2(X, C), \text{member}(X, g(X, C)) \\ \text{length}(g(X, C), C) &:- v_2(X, C) \end{aligned}$$

Suppose we replace the definitions of base relations *p* and *r* in the body of expanded query $q'(X, A)$, then we get the following rules.

$$\begin{aligned} q_2(X, A) &:- v_1(X, T), \text{setof}(Y, v_1(X, T), \text{member}(Y, f(X, T)), S), \text{sum}(S, T_1), \text{length}(S, C_1), A = \frac{T_1}{C_1} \\ q_2(X, A) &:- v_2(X, C), \text{setof}(Y, v_2(X, C), \text{member}(Y, g(X, C)), S), \text{sum}(S, T_1), \text{length}(S, C_1), A = \frac{T_1}{C_1} \end{aligned}$$

The above rules can be simplified using Γ_s as follows.

$$\begin{aligned} q_3(X, A) &:- v_1(X, T), \text{sum}(f(X, T), T), \text{length}(f(X, T), C_1), A = \frac{T}{C_1} \\ q_3(X, A) &:- v_2(X, C), \text{sum}(g(X, C), T_1), \text{length}(g(X, C), C), A = \frac{T_1}{C} \end{aligned}$$

The rules for q_3 can be further simplified by substituting the bodies of the inverse rules defining *sum* and *length*.

$$\begin{aligned} q_4(X, A) &:- v_1(X, T), \text{length}(f(X, T), C_1), A = \frac{T}{C_1} \\ q_4(X, A) &:- v_2(X, C), \text{sum}(g(X, C), T_1), A = \frac{T_1}{C} \end{aligned}$$

Unfortunately, without additional information, the query plan $\{q'\} \cup \Gamma_s \cup V^{-1}$ *cannot* fold the subgoals $\text{length}(f(X, T), C_1)$ and $\text{sum}(g(X, C), T_1)$ using views v_1 and v_2 . The information that is missing here is the relationship between the Skolem terms $f(X, T)$ and $g(X, C)$.

Consider the following rewriting of views v_1 and v_2 that is equivalent to their supplied definitions.

$$\begin{aligned} v_1(X, T) &:- v_a(X, S), \text{sum}(S, T) \\ v_2(X, C) &:- v_a(X, S), \text{length}(S, C) \\ v_a(X, S) &:- p(X), \text{setof}(Y, r(X, Y), S) \end{aligned}$$

Inverting the above views results in the following rules.

$$\begin{aligned} p(X) &:- v_a(X, S) \\ r(X, Y) &:- v_a(X, S), \text{member}(X, S) \\ v_a(X, f_1(X, T)) &:- v_1(X, T) \\ v_a(X, g_1(X, C)) &:- v_2(X, C) \\ \text{sum}(f_1(X, T), T) &:- v_1(X, T) \\ \text{length}(g_1(X, C), C) &:- v_2(X, C) \end{aligned}$$

Consider two tuples: $v_a(x, f_1(x, t))$ and $v_a(x, g_1(x, c))$. From the definition of v_a , for every binding of X such that $p(X)$ is in the extension of p , there is exactly one tuple of the form $v_a(X, S)$. Therefore, it directly follows that the sets $f_1(x, t)$ and $g_1(x, t)$ must refer to the same set.

$$\forall X, Y_1, Y_2 (v_a(X, Y_1) \wedge v_a(X, Y_2) \implies Y_1 = Y_2)$$

Incorporating functional dependencies. The functional dependency on v_a can be incorporated into our $\text{Inverse}_{\text{Agg}}$ algorithm by using the technique described in [9] that encodes the dependencies as *chase rules*. The above dependency can be captured using the following chase rule.

$$e(Y_1, Y_2) :- v_a(X, Y_1), v_a(X, Y_2)$$

A tuple of the form $e(X, Y)$ characterizes that terms X and Y are equal. To ensure that e is transitive, it suffices to include the following rules.

$$\begin{aligned} e(X, X) \\ e(X, Y) &:- e(X, Z), e(Z, Y) \end{aligned}$$

We can extend the $\text{Inverse}_{\text{Agg}}$ algorithm to incorporate chase rules in the folding of a query q by iteratively applying the following transformations that are described in [9].

1. For every constant c that appears in a subgoal of q , replace c by a variable V_c and add the subgoal $e(c, V_c)$ to q .

2. For every variable V that appears in the head of q , we replace V in the body of q by a variable V' and add the subgoal $e(V', V)$ to q .
3. For every variable V that does not appear in the head of q , and appears in two subgoals of q , replace one of the occurrences of V by a variable V' and add the subgoal $e(V', V)$ to q .

For a given query q , let us denote the query that is obtained by applying the above transformation and by expanding the list functions in q as t_q . By leveraging chase rules [9], we modify $\text{Inverse}_{\text{Agg}}$ algorithm as follows to fold a query q using views $v_1, v_2, \dots, v_n$, when the list function in the query is defined as a conjunctive view over the list functions in the views.

1. Let V_r be the rewriting of $v_1, v_2, \dots, v_n$ using the transformation described in Appendix 1.2.
2. For every view $v \in V_r$, iteratively apply the following transformations.
 - (a) For every $v \in V_r$ that contains a *setof* subgoal and list functions, rewrite v using an auxiliary view v_a such that v_a contains the *setof* subgoal but no list functions, and the rewritten view v does not contain any *setof* subgoals.
 - (b) For every $v \in V_r$ if free variables are projected out from the head of the view, then rewrite v using an auxiliary view v_p where these variables are included in head.

Let V_l denote the set of V_r after applying the above transformations.

3. Apply the $\text{Inverse}_{\text{Agg}}$ algorithm to the query t_q and views V_l . Let P be the generated query plan.
4. Encode the applicable functional dependencies as a set of chase rules E_v .

$$E_v = \{e(Y_1, Y_2) :- v_a(X, Y_1), v_a(X, Y_2)\} \cup \bigcup_{v \in V_l} S_v \cup \{e(X, X)\}$$

In the above expression, c is a constant that appears in P or in the extensions of the views. For every $v \in V_l$, if v has the general form, $v(\bar{X}, S) :- \phi(\bar{X}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$, then $S_v = \{e(S_1, S_2) :- v(\bar{X}, S_1), v(\bar{X}, S_2)\}$. Otherwise, $S_v = \{\}$.

5. The output query plan is $P \cup E_v$.

Note that the generated query plan *does not* include the definitions list functions, e.g., *sum* or *count* that appear in the views. Applying the steps 1–5 to the query folding scenario from Example 6.5.2, results in the following query plan. Note that, in the following, we have included

only the relevant e subgoals in t_q to simplify our presentation of the query plan.

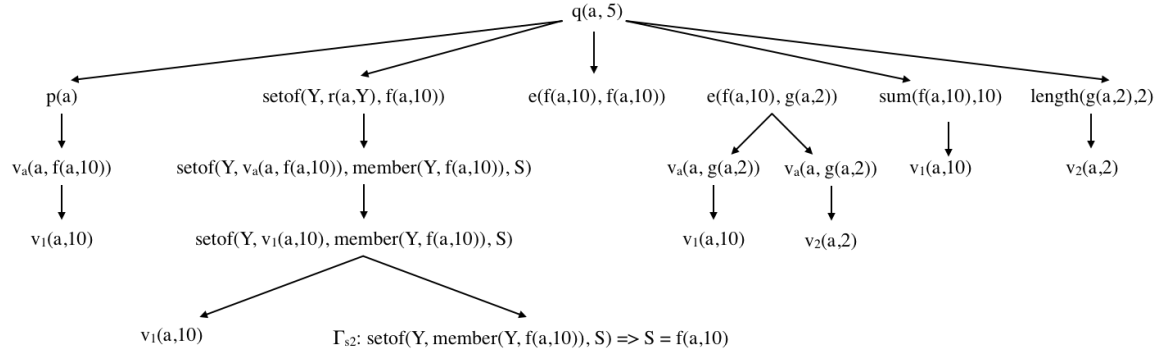
$$\begin{aligned}
 t_q(X, A) &:- p(X), \text{setof}(Y, r(X, Y), S), e(S, S_1), e(S, S_2), \text{sum}(S_1, T), \text{length}(S_2, C), A = \frac{T}{C} \\
 p(X) &:- v_a(X, S) \\
 r(X, Y) &:- v_a(X, S), \text{member}(X, S) \\
 v_a(X, f_1(X, T)) &:- v_1(X, T) \\
 v_a(X, g_1(X, C)) &:- v_2(X, C) \\
 \text{sum}(f_1(X, T), T) &:- v_1(X, T) \\
 \text{length}(g_1(X, C), C) &:- v_2(X, C) \\
 e(X, X) \\
 e(X, Y) &:- e(X, Z), e(Z, Y) \\
 e(S_1, S_2) &:- v_a(X, S_1), v_a(X, S_2)
 \end{aligned}$$


Figure 6.2: Folding Datalog_A queries using modified Inverse_{Agg} algorithm

Suppose the extensions of v_1 and v_2 consist of the tuples $v_1(a, 10)$ and $v_2(a, 2)$ respectively. In this case, the answer $q(a, 5)$ is computed using the query plan generated by the modified Inverse_{Agg} algorithm. A derivation tree of $q(a, 5)$ in this query plan is shown in Figure 6.2.

Theorem 6.5.1. *The query plan generated by the modified Inverse_{Agg} algorithm is contained in the query, if every list function that appears in the query is either identical to a list function that appears in the views or is defined as a conjunctive view over list functions in the views, and one of the following conditions holds.*

- *The supplied query is monotonic.*
- *A canonical aggregate view is included in the set of views for every relation that appears inside a setof subgoal in the query.*

Proof. In the modified $\text{Inverse}_{\text{Agg}}$ algorithm, the list functions that appear in the views are treated no different than base relations, and as a result, the containment of generated query plans follows directly from Theorems 6.4.1 and 6.4.2. $\square$

Chapter 7

Summary and Future Work

7.1 Summary

We presented an extension of Datalog, called Datalog_A that is more expressive than prior aggregate extensions of Datalog. Keys to the increased expressivity in Datalog_A are the inclusion of lists and sets as first class citizens in the language and the addition of a general aggregation operator, called *setof*, that allows us to characterize sets of objects with specified properties.

We also presented an algorithm, called CReaM, to incrementally update materialized Datalog_A views in response to changes to the underlying data.

Finally, we presented an algorithm, called Inverse_{Agg} , to fold Datalog_A queries using views, and characterized two classes of folding problems for which our algorithm generates maximally contained answers.

7.2 Future Directions

In the following, we discuss potential extensions of our work for future research.

7.2.1 Aggregates in Datalog

To ensure finite answers of Datalog_A queries, we require that programs be admissible. However, the following inadmissible query also has finite answers.

$$\begin{aligned} q(a!X) &:- p(X), q(X) \\ q(X) &:- p(X) \\ p([a]) \end{aligned}$$

What other restrictions can be applied on Datalog_A programs to ensure finite answers? Is there a significant increase in expressive power in such cases? These are open questions that require further investigation.

7.2.2 Efficient Incremental Maintenance

An interesting direction of future research in incremental maintenance of aggregate views is to explore incremental maintenance algorithms that guarantee optimality in the worst-case or for all cases. A related question is whether this algorithm leverages differential rules. A technical report on view maintenance [40] has argued, but not formally proved, that differential algorithms are not amenable to efficient incremental maintenance; as a result, the author advocates separate maintenance algorithms for each aggregate function.

Efficiency of incremental maintenance may potentially be increased by reformulating the underlying database. Database reformulation is the process of rewriting the data and rules in deductive databases in a functionally equivalent manner, ideally in ways that decrease query processing time while keeping storage costs within acceptable bounds.

We've already seen an example of database reformulation resulting in efficient maintenance. Given a view v that is defined as $v(X, S) :- p(X, Z), \text{ setof}(Y, r(X, Y), S)$, the CReaM algorithm from Chapter 5 rewrites v as follows.

$$\begin{aligned} v'(X, S) &:- v_a(X, Z, S) \\ v_a(X, Z, S) &:- p(X, Z), \text{ setof}(Y, r(X, Y), S) \end{aligned}$$

In subsequent steps, view v_a is materialized, and v_a and v' are incrementally maintained. Typically, in such scenarios, one would expect an increase in the total maintenance time due to the overhead of maintaining additional materialized views. While it may seem counter-intuitive, the time taken to incrementally maintain v' and v_a is less than the time taken to incrementally maintain v .

How can a database be reformulated such that the resulting maintenance time is minimized? This is an open question that warrants further research. Related to this question is the problem of finding Pareto-optimal database reformulations that minimize query evaluation and update times. A recent result presented in [11] discusses a solution to the optimal database reformulation problem that minimizes query evaluation times with at most linear growth in storage space.

7.2.3 Query Folding

In this thesis, we have identified classes of query folding problems for which our $\text{Inverse}_{\text{Agg}}$ rules algorithm generates maximally contained query plans. The problem of determining existence of

equivalent query plans for Datalog queries using Datalog views has already been shown to be undecidable [8]. That said, it is an open question whether we can characterize non-trivial classes of query folding problems in Datalog_A where an equivalence rewriting is guaranteed to exist.

Another direction to extend our work on inverting Datalog_A views would be to deal with inversion of list functions. Naively, this sort of inversion results in non-terminating and inadmissible query plans. In some cases, however, it may be possible to determine an upper-bound on the depth of a recursive call. For example, in a graph with n -vertices, an acyclic path between any two nodes may be of length at most $n - 1$.

Appendix A

Proofs

A.1 Proof of Lemma 3.3.1

Proof. First, we present a transformation to unnest nested *setof* subgoals in a rule. Then, we present a transformation to rewrite rules with multiple unnested *setof* subgoals using rules that contain at most one *setof* subgoal.

Consider a view v that is defined over the formulas $\phi(\bar{X}, \bar{Y})$ and $\psi(\bar{X}, \bar{Z})$ as follows.

$$v(\bar{X}, S) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$$

In the above definition of v , ψ may contain additional, and potentially nested *setof* subgoals. Algorithm 1.1 described below generates an equivalent rewriting of v using rules that contain *at most* one *setof* subgoal such that for all databases D , the extensions of v and its rewriting over D are identical.

Algorithm 1.1 Flattening rules containing nested *setof* subgoals

Input: Rule containing nested *setof* subgoals $v(\bar{X}, S) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)$

Initialize $V \leftarrow \{\}$

if ψ *does not* contain *setof* subgoals **then**

$V = \{v(\bar{X}, S) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, \psi(\bar{X}, \bar{Z}), S)\}$

else

Define a new relation v_x as: $v_x(\bar{X}, \bar{Z}) :- \phi(\bar{X}, \bar{Y}), \psi(\bar{X}, \bar{Z})$

$V_x \leftarrow$ output of recursively applying this algorithm on v_x .

$V = V_x \cup \{v(\bar{X}, S) :- \phi(\bar{X}, \bar{Y}), \text{setof}(\bar{Z}, v_1(\bar{X}, \bar{Z}), S)\}$

end if

Output: V

Consider a view v that is defined as: $v := s_1, s_2 \dots s_n$. Algorithm 1.2 described below redefines v as the view v' using new views $v_1, v_2, \dots v_{|V|}$, each of which contain *at most* one *setof* subgoal. Unfolding v' results in $s_1, s_2 \dots s_n$, proving that the views v and v' are equivalent.

Algorithm 1.2 Redefining views using rules that contain at most one *setof* subgoal

Input: View definition $v := s_1, s_2 \dots s_n$

Initialize $i \leftarrow 1, j \leftarrow 1, V \leftarrow \{\}, S \leftarrow \{\}$

while $j < n$ **do**

if $i > 1$ **then**

$V \leftarrow V \cup \{v_i := v_{i-1}, s_i\}$

else

$V \leftarrow V \cup \{v_i := s_i\}$

end if

$j \leftarrow j + 1$

end while

Output: $V \cup \{v' := v_1, \dots, v_{|V|}\}$

□

A.2 Analysis of Worst-Case Time Complexity of Differential Rules

Case 1: The underlying dataset has relational indexing. For this case, we discuss the number of computations (t) required to compute the differential relations v^+ and v^- for $\Delta_1 - \Delta_5$ below.

- $t(\Delta_1) = c + c \times n_v = O(n_v)$. Time taken taken to evaluate the output of this differential rule is computed as the time taken to look up $\phi^-(\bar{X}) = O(1) = c$, and the time taken to look up $v(\bar{X}, S)$ for each $\phi^-(\bar{X}) = c \times n_v$.
- $t(\Delta_2) = c + c \times n_\psi \times \log n_\psi + c \times c \times \log c + c \times c \times \log c = O(n_\psi \times \log n_\psi)$.
- $t(\Delta_3) = 2 \times c + 2 \times c \times n_v + 2 \times c \times c + 2 \times c \times c + 2 \times c \times c \times \log c + 2 \times c \times c \times \log c = O(n_v)$.
- $t(\Delta_4) = c + c \times n_v = O(n_v)$.
- $t(\Delta_5) = c + c \times n_v = O(n_v)$.

Analogously, we compute the worst case time complexity for $\Gamma_1 - \Gamma_5$ below.

- $t(\Gamma_1) = c + c \times n_v + c \times (c + n_\phi + n_\phi \times c) = O(n_v + n_\phi)$.

- $t(\Gamma_2) = c + c \times n_v + c \times n_\psi \times \log n_\psi + c \times c \times \log c + c \times c \times \log c = O(n_v + n_\psi \times \log n_\psi)$.
- $t(\Gamma_3) = 2 \times c + 2 \times c \times n_v + 2 \times c \times (c + n_\phi + n_\phi \times c) + 2 \times c \times c \times \log c + 2 \times c \times c \times \log c = O(n_v + n_\phi)$.
- $t(\Gamma_4) = c + c \times n_v = O(n_v)$.
- $t(\Gamma_5) = c + c \times n_v = O(n_v)$.

Case 2: The underlying dataset has full indexing. For this case, we discuss the number of computations (t) required to compute the differential relations v^+ and v^- for $\Delta_1 - \Delta_5$ below.

- $t(\Delta_1) = c + c \times \log n_v = O(\log n_v)$. Time taken to evaluate the output of this differential rule is computed as the time taken to look up $\phi^-(\bar{X}) = O(1) = c$, and the time taken to look up $v(\bar{X}, S)$ for each $\phi^-(\bar{X}) = c \times \log n_v$.
- $t(\Delta_2) = c + c \times n_\psi \times \log n_\psi + c \times c \times \log c + c \times c \times \log c = O(n_\psi \times \log n_\psi)$.
- $t(\Delta_3) = 2 \times c + 2 \times c \times \log n_v + 2 \times c \times c + 2 \times c \times c + 2 \times c \times c \times \log c + 2 \times c \times c \times \log c = O(\log n_v)$.
- $t(\Delta_4) = c + c \times \log n_v = O(\log n_v)$.
- $t(\Delta_5) = c + c \times \log n_v = O(\log n_v)$.

Analogously, we compute the worst case time complexity for $\Gamma_1 - \Gamma_5$ below.

- $t(\Gamma_1) = c + c \times \log n_v + c \times (c + \log n_\phi + n_\phi \times c) = O(\log n_v + n_\phi)$.
- $t(\Gamma_2) = c + c \times \log n_v + c \times (c + \log n_\phi + n_\phi \times c) = c + c \times \log n_v + c \times n_\psi \times \log n_\psi + c \times c \times \log c + c \times c \times \log c = O(\log n_v + n_\psi \times \log n_\psi)$.
- $t(\Gamma_3) = 2 \times c + 2 \times c \times \log n_v + 2 \times c \times (c + \log n_\phi + n_\phi \times c) + 2 \times c \times c \times \log c + 2 \times c \times c \times \log c = O(\log n_v + n_\phi)$.
- $t(\Gamma_4) = c + c \times \log n_v = O(\log n_v)$.
- $t(\Gamma_5) = c + c \times \log n_v = O(\log n_v)$.

Appendix B

List Functions

- Members of a list

$$\begin{aligned} &member(X, X!Y) \\ &member(Z, X!Y) :- member(Z, Y) \end{aligned}$$

- First element of a list

$$first(X, X!Y)$$

- Last element of a list

$$\begin{aligned} &last(X, [X]) \\ &last(Z, X!Y) :- last(Z, Y) \end{aligned}$$

- Reverse of a list

$$\begin{aligned} &reverse(X, Y) :- revappend(X, [], Y) \\ &revappend([], Y, Y) \\ &revappend(X!L, Y, Z) :- revappend(L, X!Y, Z) \end{aligned}$$

- Length of a list

$$\begin{aligned} &length([], 0) \\ &length(X!Y, C) :- length(Y, A), C = A + 1 \end{aligned}$$

- **Sum of a list**

$$\begin{aligned} &sum([], 0) \\ &sum(X!Y, C) :- sum(Y, A), C = A + X \end{aligned}$$

- **Average of a list**

$$avg(X, A) :- sum(X, S), length(X, C), A = \frac{S}{C}$$

- **Maximum element of a list**

$$\begin{aligned} &max([X], X) \\ &max(X!Y, X) :- max(Y, M), X \geq M \\ &max(X!Y, M) :- max(Y, M), X < M \end{aligned}$$

- **Minimum element of a list**

$$\begin{aligned} &min([X], X) \\ &min(X!Y, X) :- max(Y, M), X < M \\ &min(X!Y, M) :- max(Y, M), X \geq M \end{aligned}$$

- **Bag-Union of two lists**

$$\begin{aligned} &bagunion(X, [], X) \\ &bagunion(X, Y!Z, Y!W) :- bagunion(X, Z, W) \end{aligned}$$

- **Set-Union of two lists**

$$\begin{aligned} &union(X, [], X) \\ &union(X, Y!Z, Y!W) :- union(X, Z, W), \neg member(Y, W) \\ &union(X, Y!Z, W) :- union(X, Z, W), member(Y, W) \end{aligned}$$

- Difference of two lists

$$\begin{aligned} & \text{difference}([], X, []) \\ & \text{difference}(X!Y, Z, X!W) :- \neg \text{member}(X, Z), \text{difference}(Y, Z, W) \\ & \text{difference}(X!Y, Z, W) :- \text{member}(X, Z), \text{difference}(Y, Z, W) \end{aligned}$$

- Sub-lists of a list

$$\begin{aligned} & \text{sublist}(X, []) \\ & \text{sublist}(X!Y, Z) :- \text{sublist}(Y, Z) \\ & \text{sublist}(X!Y, X!Z) :- \text{sublist}(Y, Z) \end{aligned}$$

- Permutations of a list

$$\begin{aligned} & \text{permute}([], []) \\ & \text{permute}(X!Y, Z) :- \text{permute}(Y, W), \text{place}(X, W, Z) \\ & \text{place}(X, [], [X]) \\ & \text{place}(X, Y!Z, X!Y!Z) \\ & \text{place}(X, Y!Z, Y!W) :- \text{place}(X, Z, W) \end{aligned}$$

- Collection of first elements of a nested-list

$$\begin{aligned} & \text{collectfirst}([], []) \\ & \text{collectfirst}([Y, Z]!T, Y!S) :- \text{collectfirst}(T, S) \end{aligned}$$

Bibliography

- [1] Foto Afrati and Rada Chirkova. Selecting and using views to compute aggregate queries. In *Proceedings of the 10th International Conference on Database Theory*, pages 383–397. Springer-Verlag, 2005.
- [2] K. R. Apt and H. A. Blair. Arithmetic classification of perfect models of stratified programs. *Fundamenta Informaticae*, 13(1):1–17, 1990.
- [3] Leopoldo E. Bertossi, Jan Chomicki, Alvaro Cortés-Calabuig, and Claudio Gutiérrez. Consistent answers from integrated data sources. In *Flexible Query Answering Systems*, pages 71–85. Springer Berlin Heidelberg, 2002.
- [4] S. Ceri, G. Gottlob, and L. Tanca. What you always wanted to know about datalog (and never dared to ask). *IEEE Transactions on Knowledge and Data Engineering*, 1(1):146–166, 1989.
- [5] W. F. Clocksin and Christopher S. Mellish. *Programming in Prolog (2nd Ed.)*. Springer-Verlag New York, Inc., 1984.
- [6] Sara Cohen, Werner Nutt, and Alexander Serebrenik. Rewriting aggregate queries using views. In *Proceedings of the 18th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, pages 155–166. ACM, 1999.
- [7] Mariano P. Consens and Alberto O. Mendelzon. Low complexity aggregation in graphlog and datalog. In *Proceedings of the Third International Conference on Database Theory on Database Theory*, ICDT '90, pages 379–394. Springer-Verlag New York, Inc., 1990.
- [8] Oliver M. Duschka and Michael R. Genesereth. Answering recursive queries using views. In *Proceedings of the 16th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, pages 109–116. ACM, 1997.
- [9] Oliver Michael Duschka. *Query Planning and Optimization in Information Integration*. PhD thesis, Stanford University, Stanford, CA, USA, 1998.

- [10] Ronald Fagin, Phokion G Kolaitis, Renée J Miller, and Lucian Popa. Data exchange: semantics and query answering. *Theoretical Computer Science*, 336(1):89–124, 2005.
- [11] Michael R. Genesereth and Abhijeet Mohapatra. Practical reformulation of deductive databases. In *2nd IEEE International Conference on Artificial Intelligence and Knowledge Engineering, AIKE 2019, Sardinia, Italy, June 3-5, 2019*, pages 65–72, 2019.
- [12] Todd J Green, Grigoris Karvounarakis, Zachary G Ives, and Val Tannen. Update exchange with mappings and provenance. In *Proceedings of the 33rd International Conference on Very Large Data Bases*, pages 675–686. VLDB Endowment, 2007.
- [13] Timothy Griffin and Leonid Libkin. Incremental maintenance of views with duplicates. *SIGMOD Record*, 24(2):328–339, 1995.
- [14] Ashish Gupta, Venky Harinarayan, and Dallan Quass. Aggregate-query processing in data warehousing environments. In *Proceedings of the 21st International Conference on Very Large Data Bases*, pages 358–369. Morgan Kaufmann Publishers Inc., 1995.
- [15] Ashish Gupta, Inderpal Singh Mumick, and V. S. Subrahmanian. Maintaining views incrementally. *SIGMOD Record*, 22(2):157–166, 1993.
- [16] Himanshu Gupta and Inderpal Singh Mumick. Incremental maintenance of aggregate and outerjoin expressions. *Information Systems*, 31(6):435–464, 2006.
- [17] Alon Y. Halevy. Answering queries using views: A survey. *The VLDB Journal*, 10(4):270–294, 2001.
- [18] Joseph M. Hellerstein. The declarative imperative: Experiences and conjectures in distributed logic. *SIGMOD Record*, 39(1):5–19, 2010.
- [19] Nam Huyn. Efficient view self-maintenance. In *Workshop on Materialized Views: Techniques and Applications*, pages 17–25, 1996.
- [20] Andrew David Irvine and Harry Deutsch. Russell’s paradox. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, winter 2016 edition, 2016.
- [21] Maurizio Lenzerini. Data integration: A theoretical perspective. In *Proceedings of the 21st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, pages 233–246. ACM, 2002.
- [22] Boon Thau Loo, Tyson Condie, Minos Garofalakis, David E Gay, Joseph M Hellerstein, Petros Maniatis, Raghu Ramakrishnan, Timothy Roscoe, and Ion Stoica. Declarative networking:

- language, execution and optimization. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*, pages 97–108. ACM, 2006.
- [23] Boon Thau Loo, Tyson Condie, Joseph M Hellerstein, Petros Maniatis, Timothy Roscoe, and Ion Stoica. Implementing declarative overlays. *ACM SIGOPS Operating Systems Review*, 39(5):75–90, 2005.
- [24] Boon Thau Loo, Joseph M Hellerstein, Ion Stoica, and Raghu Ramakrishnan. Declarative routing: extensible routing with declarative queries. *ACM SIGCOMM Computer Communication Review*, 35(4):289–300, 2005.
- [25] Nathaniel Love and Michael Genesereth. Computational law. In *Proceedings of the 10th International Conference on Artificial Intelligence and Law*, pages 205–209. ACM, 2005.
- [26] Mirjana Mazuran, Edoardo Serra, and Carlo Zaniolo. Extending the power of datalog recursion. *The VLDB Journal*, 22(4):471–493, 2013.
- [27] Abhijeet Mohapatra and Michael Genesereth. Aggregation in datalog under set semantics. Technical report, Logic Group, Stanford University, 2012. <http://logic.stanford.edu/reports/LG-2012-01.pdf>.
- [28] Abhijeet Mohapatra and Michael Genesereth. An incremental algorithm to optimally maintain aggregate views. In *LPAR-19. 19th International Conference on Logic for Programming, Artificial Intelligence and Reasoning*, volume 26, pages 88–96. EasyChair, 2013.
- [29] Abhijeet Mohapatra and Michael Genesereth. Reformulating aggregate queries using views. In *Symposium on Abstraction, Reformulation and Approximation*, 2013.
- [30] Abhijeet Mohapatra and Michael Genesereth. Incremental maintenance of aggregate views. In *Proceedings of the 8th International Symposium on Foundations of Information and Knowledge Systems - Volume 8367*, pages 399–414. Springer-Verlag New York, Inc., 2014.
- [31] Inderpal Singh Mumick, Hamid Pirahesh, and Raghu Ramakrishnan. The magic of duplicates and aggregates. In *Proceedings of the 16th International Conference on Very Large Data Bases*, pages 264–277. Morgan Kaufmann Publishers Inc., 1990.
- [32] Inderpal Singh Mumick, Dallan Quass, and Barinderpal Singh Mumick. Maintenance of data cubes and summary tables in a warehouse. *SIGMOD Record*, 26(2):100–111, 1997.
- [33] Inderpal Singh Mumick and Oded Shmueli. How expressive is stratified aggregation? *Annals of Mathematics and Artificial Intelligence*, 15(3):407–435, 1995.
- [34] Levent V. Orman. Differential relational calculus for integrity maintenance. *IEEE Transactions on Knowledge and Data Engineering*, 10(2):328–341, 1998.

- [35] Xiaolei Qian. Query folding. In *Proceedings of the Twelfth International Conference on Data Engineering*, pages 48–55, Washington, DC, USA, 1996. IEEE Computer Society.
- [36] Dallan Quass. Maintenance expressions for views with aggregation. In *Workshops on Materialized Views: Techniques and Applications*, pages 110–118, 1996.
- [37] Divesh Srivastava, Shaul Dar, H. V. Jagadish, and Alon Y. Levy. Answering queries with aggregation using views. In *Proceedings of the 22nd International Conference on Very Large Data Bases*, pages 318–329. Morgan Kaufmann Publishers Inc., 1996.
- [38] Jeffrey D. Ullman. *Principles of Database and Knowledge-base Systems, Vol. I*. Computer Science Press, Inc., 1988.
- [39] Jeffrey D. Ullman and Allen Van Gelder. Efficient tests for top-down termination of logical rules. *J. ACM*, 35(2):345–373, 1988.
- [40] Todd L. Veldhuizen. Incremental maintenance for leapfrog triejoin. *CoRR*, abs/1303.5313, 2013. <http://arxiv.org/abs/1303.5313>.